

Evidence-Backed Accusation: Third-Party Re-verifiable Misbehavior Alerts for A2A Agents

Author names, affiliations and corresponding address to be inserted before submission.

Abstract

In cross-organizational A2A collaboration, an agent that observes a violation cannot make its peers independently believe what it saw. Prior work turns behavior into verifiable evidence but not into a third-party re-verifiable accusation. Credential-level revocation cannot express behavioral distrust, and the protocol offers no structured way to reject unsupported alerts, so Sybil identities can manufacture false accusations and pressure honest agents into isolation. We present Evidence-Backed Accusation (EBA): two additive A2A extension objects, the signed hash-chained AuditEvent, whose signing key is generated inside the sender's TEE and never shared with its peer, and the RevokeAdvice alert, together with a deterministic three-state acceptance semantics that a party which never participated in the session can execute. Each session is anchored by a co-signed anchor record. Participants publish signed Merkle roots of the chains they hold. An alert carries an anchor reference, event position, rule version, copy hash, and inclusion proof, so a verifier needs no local copy of the chain in order to verify the evidence. A verifier returns ACCEPT, REJECT, or UNCONFIRMED: it accepts only when the anchor verifies and is fresh, the event verifies under the sender's TEE-attested session key, the inclusion proof ties the event to an attested root, the named rule version re-evaluates to a violation, and the accuser signature and evidence-copy hash are consistent. It returns UNCONFIRMED when evidence or roots are temporarily unavailable, never defaulting to acceptance, and returns REJECT when a verified object deterministically fails a condition. Fabricated, altered, mislabeled, and unsupported alerts cannot be accepted; deterministic failures return REJECT, while unavailable or malformed candidates remain UNCONFIRMED until expiry; Sybil identities can increase attempts and cost, but cannot increase the per-alert probability of acceptance. We define sessions, anchors, chains, rules, observability classes, third-party verification, and three-state acceptance; prove evidence integrity and attributability by reduction, and acceptance soundness and third-party verdict determinism by logical and deterministic analysis; provide a ProVerif model with R1-R5 and N1-N6; and report completed E0-E8 experiments covering consistency, tampering, attribution, framing, third-party verification, availability, overhead, formal verification, and payload/object privacy. The guarantee covers O1/O2 violations and assumes one honest participant per session that publishes roots and serves evidence; O3 content-invisible violations, quarantine, recovery, and response policy are out of scope. The privacy claim is limited to payload and protocol-object privacy, not anonymity or interaction-graph hiding. Hardware TEE attestation is a deployment assumption A3; the evaluation uses a software-simulated attestation interface.

Keywords: A2A protocol; verifiable evidence; behavioral accusation; third-party verification; framing resistance; three-state acceptance; ProVerif

1 Introduction

Autonomous agents increasingly act for users and organizations through machine-to-machine delegation. The Agent-to-Agent (A2A) protocol [1] is emerging as the horizontal communication layer for this activity, while MCP connects agents to tools and data. A2A builds security on TLS [2] and standard web authentication, and deliberately leaves authorization models, key distribution, and evidence handling to implementers. Correct authorization, however, does not imply trustworthy behavior: a semantically hijacked agent may hold valid keys, pass every authentication check, and still violate its principal's policy.

Consider a procurement workflow in which buyer-side agent A delegates to B, which subcontracts to C. An attacker injects an instruction into content that B reads [3]. B's keys are intact and its TEE [4] is untouched,

yet B attempts a transfer to an account outside the buyer’s allowlist. C’s gateway can detect the attempt if the violation is visible in machine-readable metadata. What C cannot do is tell D, E, and the rest of the federation, in a way they can independently verify, that B misbehaved on that interaction.

Credential revocation in the standard X.509 sense, including certificate revocation lists [5] and OCSP [6], is the wrong instrument: B’s credentials are valid, and revocation cannot express "B’s behavior is untrustworthy on this task." A reputation score is equally unsuitable because agent identity and behavior are loosely coupled [7].

Prior work has solved the evidence half of the problem. VET [8] generates verifiable execution traces from TEE attestations, succinct proofs, or notarized TLS. VCT [9] protects conversation transcripts with branch hash chains [10], session Merkle roots [11], and dual signatures. Agent-Native Telemetry [12] signs hash-chained state deltas behind digest-verified references. A2Auth [13] binds capabilities with certificates and maintains a hash-linked bilateral ledger for post-hoc forensics. Verifiable manifest signing [14] enforces policy-bound tool-use manifests. These systems answer "who did what, and can we prove that the record was not altered." None answers "who may accuse whom, and what makes an accusation credible to a third party."

The second, less discussed half is that the protocol offers no way to reject an unsupported accusation. In a permissionless federation, any participant can accuse any other. If acceptance depends on the receiver’s judgement, Sybil identities [15] can flood alerts until honest agents quarantine their peers, and the resulting denial of service is indistinguishable from legitimate enforcement. Governance analyses [16] and open-challenge surveys [17] list membership admission, containment, attribution, and provenance as unsolved.

This paper asks one narrow question: without a central arbiter, and against an adversary that controls the network and can create many accusing identities, when should an honest verifier accept an alert about another agent? We answer with a deterministic rule whose inputs are public evidence and public rules, never the accuser’s reputation.

1.1 Contributions

1. **Two additive A2A extension objects.** AuditEvent binds session, direction, sequence, participants, behavior metadata, permission scope, payload hash, and chain predecessor. RevokeAdvice binds an anchor reference, event position, rule identity and version, rule re-evaluation result, event-copy hash, inclusion proof, and accuser signature.
2. **A deterministic three-state acceptance semantics for participants and non-participants.** A verifier returns ACCEPT, REJECT, or UNCONFIRMED. Deterministic failure takes priority over unavailability. Missing evidence or roots produce UNCONFIRMED until the TTL expires. Except for signature validity, the acceptance predicate contains no term in the accuser’s identity or reputation.
3. **A commitment and retrieval mechanism.** The design uses per-direction hash chains, session anchors, signed Merkle roots, inclusion proofs, and an evidence-retrieval protocol. A single attested root supports event verification; multiple roots over the same range support truncation and equivocation detection.
4. **A security analysis.** T1 proves evidence integrity; T2 proves attributability; T3 proves acceptance soundness and framing resistance; T4 proves third-party verdict determinism. P1/P2 separate availability safety from conditional liveness, and PR1 defines the payload-privacy boundary. T1 and T2 are established through cryptographic reductions; T3 and T4 are established by logical and deterministic proofs over the acceptance predicate.
5. **A completed falsifiable evaluation.** E0 tests consistency and interoperability; E1 evidence integrity; E2 attribution; E3 framing resistance; E4 third-party verdict determinism; E5 availability; E6 local performance; E7 ProVerif R1-R5 and N1-N6; and E8 payload and protocol-object privacy. The reported results use pass and failure conditions fixed before execution.

The remainder of the paper reviews related mechanisms in Section 2, defines the system and threat model in Section 3, presents the method in Section 4, proves security in Section 5, reports the evaluation in Section

6, discusses limitations in Section 7, concludes in Section 8, and provides declarations and appendices in Section 9 and Appendices A-E.

2 Background and Related Work

2.1 A2A and Its Security Surface

A2A defines a canonical data model, abstract operations, and protocol bindings (JSON-RPC, gRPC, HTTP+JSON/REST, and custom bindings). Discovery uses a JSON Agent Card whose security-scheme objects declare authentication. The specification leaves authorization models, key distribution, and evidence handling to implementations.

A2ABreak [18] extracts a finite-state machine from the specification and enumerates protocol-level attacks. Improving Google A2A [19] identifies weaknesses in sensitive-data settings and repairs authorization governance with an interceptor architecture. A comparative evaluation [20] applies a vulnerability taxonomy across A2A, ACP [21], and CORAL. This group establishes where the protocol is weak, but it does not produce a behavioral accusation that third parties can re-verify.

2.2 Evidence, Accountability, and Accusation

VET [8], VCT [9], Agent-Native Telemetry [12], and A2Auth [13] produce execution traces, authenticated transcripts, signed state chains, and a bilateral ledger, respectively. Delegation-ancestry attestation [22] binds outputs to co-signed delegation DAGs and states that signatures cannot prevent prompt injection [3]. Verifiable manifest signing [14] signs tool-use manifests and enforces them with a transparency log. Tamper-evident logging systems such as SealFSv2 [23] and secure audit logs [24] supply immutability mechanisms for the chain itself.

These systems answer whether a record exists, whether it was altered, and who did what. They do not define what makes an alert acceptable. This paper does not replace evidence-generation mechanisms. It defines the interface after evidence generation: how a party that did not participate in the session, and that trusts neither the accuser nor the accused, obtains a reproducible ACCEPT, REJECT, or UNCONFIRMED verdict from public anchors, attested roots, evidence bundles, and rule versions.

Transparency logging [25] detects equivocation of published statements through an append-only log and monitors, but its subject is certification rather than behavior. An entry attests that something was published, not that an agent violated a rule. Non-repudiation protocols [26] give origin and receipt evidence for individual messages with a trusted third party, but our setting has no arbiter. PeerReview [27] proves Byzantine faults from tamper-evident logs and challenge-response among participating nodes, but it does not define how a party that distrusts both accuser and accused re-evaluates a cross-domain rule, nor how many accusing identities should be discounted.

2.3 Identity, Delegation, and Trust Frameworks

InterSAGE [28], AIP [29], LDP [30], BlockA2A [31], Context Lineage [32], and AINA'26 [33] provide identity, delegation, capability, provenance, or zero-trust policy mechanisms. They answer who an agent is, what it may do, and where it came from, with revocation that is credential- or authorization-level. They do not answer how a behavioral violation is accused.

This paper does not introduce a new identity-issuance primitive. It consumes an existing identity binding through A0: the long-term identity key IK_i is uniquely bound to agent identity i by AgentCard, verifiable credentials, SPIFFE, or another deployment mechanism. The TEE quote and session anchor bind the session key to IK_i , while A0 connects that key to the real-world identity. This separates cryptographic attributability from real-world identity attribution.

2.4 Runtime Defense and Monitoring

A2AS [34], Verifiability-First Agents [35], Alignment Contracts [36] with their enforcement-soundness basis [37], and ACLE-MCP [38] gate or monitor a single agent’s execution. ClawGuard [39] and AgentWatcher [40] add out-of-band detection. These mechanisms may detect a violation locally, but they provide no distributed, evidence-anchored way for peers to act on it.

2.5 Closest-Work Comparison

Work	Mechanism	Leaves Open
InterSAGE	Identity, capability attenuation, hash-linked traces	No accusation primitive; verification deferred
A2Auth	Capability certificates, reproducible replay, bilateral ledger	Runtime attacks are post-hoc only; certificate-level revocation
Delegation Ancestry	Co-signed delegation DAGs, output hashes	Cannot prevent injection; no targeted revocation
VET / VCT	Execution traces, branch chains, Merkle roots, dual signatures	Evidence without accusation semantics; conversation-scoped
Agent-Native Telemetry	Signed state chains, opaque text references	No cross-peer accusation rule
Transparency logging	Append-only log of signed statements, monitors	Certification subject; log and monitor trust assumptions
Non-repudiation protocols	Origin and receipt evidence, fair exchange	Trusted third party; no rule re-evaluation or Sybil discounting
Revocation, CRLs, OCSP	Signed revocation lists and certificate status responses	No behavioral record; requires a CA or responder
PeerReview	Tamper-evident logs, challenge-response proofs	Replicated-service proofs; assumes correct nodes log and answer
LDP / BlockA2A / AINA’26	Identity cards, verifiable credentials, SPIFFE, policy-as-code	No alert primitive; centralized or credential-level response
This paper	AuditEvent + RevokeAdvice + anchors + attested roots + three-state deterministic acceptance + retrieval	O3 invisible violations; availability A6; rule availability A9; payload/object privacy only; no quarantine, recovery, or response policy

2.6 Gap Statement

Evidence exists, identity exists, and log-based equivocation detection exists, but the connective tissue is missing. No primitive converts a violation observed by one honest participant into a claim that a party distrusting both accuser and accused can independently re-verify. No mechanism prevents unsupported claims from being accepted when accusing identities are free to create. At the same time, acceptance must not confuse temporary unavailability with invalid evidence, and it must never treat "not yet verified" as acceptance.

This paper defines that primitive, its ACCEPT/REJECT/UNCONFIRMED acceptance rule, and the anchors, roots, inclusion proofs, and retrieval path that let a non-participant execute it. The response that follows an accepted alert, including containment and recovery, is outside the scope of this paper.

3 System and Threat Model

3.1 Roles, Identity Binding, and Assumptions

The system contains agents $A1..An$. Each agent is operated by a deployer and runs a protocol gateway between the network and its LLM context. The gateway performs signature verification, chain validation, policy evaluation, alert handling, root publication, evidence retrieval, and content admission. All checks precede admission, so content that fails a check never reaches the LLM context.

Each agent i holds a long-term identity key IK_i . This paper does not redesign identity issuance. It consumes an existing identity mechanism, such as AgentCard, verifiable credentials, or SPIFFE, through A0: the identity provider uniquely binds IK_i to agent identity i , honest verifiers resolve the binding, and the binding remains stable for the session.

For every session s , the gateway generates a fresh session signing key pair $(sk_{i,s}, pk_{i,s})$ inside its TEE. The TEE issues a quote q_i [41] binding $(IK_i, pk_{i,s}, sid, T_{start}, T_{end})$. The session private key is never exported, shared, or derivable by the peer. What two participants share is the transcript and public verification material, not a signing secret. Per-party session keys are not an optimization; they are the property that makes T2 non-vacuous.

An operator configures rules and thresholds but does not arbitrate accusations. There is no central arbiter and no trusted storage. Logs, gossip channels, and evidence providers are untrusted for soundness.

3.2 Adversary

The adversary controls the semantic layer of a subset of agents: it chooses the content they send, subject to their protocol wrappers. It controls the network under Dolev-Yao [42], may create Sybil agents, may coordinate accusations from agents it controls, and may withhold evidence or refuse to publish roots. It does not possess IK_i or $sk_{i,s}$, cannot forge a TEE quote, cannot alter gateway logic, and cannot break Ed25519 [43], [44] or SHA-256.

Key compromise, gateway compromise, physical TEE attacks, and TEEs that export session keys are out of scope.

3.3 Observability Classes

O1 (metadata-visible): the violation is decidable from machine-readable fields such as permission scope, delegation depth, call rate, and target identity, without inspecting payload content.

O2 (receiver-visible): the violation is decidable only by the direct receiver through a content policy applied to the message.

O3 (invisible): neither metadata nor receiver policy flags the message; the adversarial instruction is entirely inside semantically compliant content.

This paper guarantees evidence, accusation, and verification properties for O1 and O2 only.

3.4 Formal Definitions

D1 (Session and Keys). A session s between a and b consists of an identifier sid , a monotone sequence counter per direction, and, for each $i \in \{a, b\}$, a session signing key pair $(sk_{i,s}, pk_{i,s})$ generated inside i 's TEE. Directions are $d \in \{a \rightarrow b, b \rightarrow a\}$. Direction $a \rightarrow b$ carries the events a signs for b . Neither peer obtains the other's private key.

$$(sk_{i,s}, pk_{i,s}) \leftarrow \text{KeyGen}_{\text{TEE}}(i, s), \quad (1)$$

$$q_i \leftarrow \text{Quote}_{\text{TEE}}(i, s, sid, T_{start}, T_{end}), \quad (2)$$

$$K_{\text{enc}} = \text{HKDF}(g^{xy}, sid). \quad (3)$$

K_{enc} protects transport only and is never used for event authentication or alert acceptance.

D2 (Session Anchor).

$$A_s = (sid, IK_a, pk_{a,s}, IK_b, pk_{b,s}, h_{a \rightarrow b}^0, h_{b \rightarrow a}^0, T_{\text{start}}, T_{\text{end}}, \sigma_a, \sigma_b), \quad (4)$$

where h_d^0 is the initial head of direction d and $\sigma_i = \text{Sign}_{IK_i}(\text{JCS}(\text{preceding fields}))$. Its reference is $\text{anchor_ref} = H(\text{JCS}(A_s))$. The anchor states which identities, keys, directions and validity windows are jointly committed.

D3 (AuditEvent and Evidence Chain).

$$e = (sid, d, seq, ts, sender, receiver, intent_meta, perm_scope, payload_hash, prev_hash, \sigma), \quad (5)$$

$$\sigma = \text{Sign}_{sk_{\text{sender},s}}(\text{JCS}(\text{preceding fields})), \quad (6)$$

$$e_{d,1}.prev = h_d^0, \quad e_{d,i}.prev = H(e_{d,i-1}) \quad (i \geq 2). \quad (7)$$

Here $sender$ is consistent with direction d . For direction d , the chain is $e_{d,1}..e_{d,n}$ with $e_{d,1}.prev = h_{0,d}$ and $e_{d,i}.prev = H(e_{d,i-1})$.

Each participant holds an outbound chain and an inbound chain. The outbound chain of a is the authoritative record of a 's behavior in direction $a \rightarrow b$; the inbound chain of b is b 's admitted copy.

D4 (Rule, Version, and Resolvability). A rule is a deterministic predicate $r : \text{Event} \rightarrow \{\text{ok}, \text{violation}\}$ over machine-readable fields. It is identified by rule_id and rule_version and resolved through rule_ref to the canonical rule content. A verifier must re-evaluate the referenced version locally. It may not substitute the latest rule, a default rule, or an approximate version.

A9 (rule availability): any rule version cited by an accepted alert remains resolvable for the evidence-retention horizon. The publication system provides the rule content and version hash.

If a verifier lacks the referenced version, it returns $\text{REJECT}(\text{RULE_UNKNOWN})$. The rule set and thresholds are deployment configuration. This paper guarantees deterministic execution, not rule correctness.

D5 (In-Session Event Admission). The receiver v of direction d admits e if and only if:

1. the credential and sender quote verify against the anchor;
2. σ verifies under the session public key that the anchor binds to $e.\text{sender}$, and $e.\text{sender}$ agrees with d ;
3. $e.\text{prev_hash}$ equals v 's stored head for d , with out-of-order events buffered until their predecessor arrives;
4. $e.\text{seq}$ is unused in d ;
5. $e.\text{ts}$ lies inside the anchor window within the clock-skew allowance; and
6. $\text{Policy}(e.\text{intent_meta}, e.\text{perm_scope}, e) = \text{ok}$.

D5 outputs whether the event is admitted. It is not the final alert verdict. Successful D5 produces $\text{ValidEvent}(e)$. Failure produces a typed in-session rejection and does not advance the head.

D6 (RevokeAdvice Alert).

$$a = (\text{accuser}, ts, \text{anchor_ref}, d, seq, \text{rule_id}, \text{rule_version}, \text{result}, \text{copy_hash}, \text{root_ref}, \pi, \sigma_{\text{accuser}}), \quad (8)$$

where $\text{copy_hash} = H(\text{JCS}(e))$, root_ref names an attested Merkle root under which e is included, and $\sigma_{\text{accuser}} = \text{Sign}_{IK_{\text{accuser}}}(\text{JCS}(\text{preceding fields}))$. An alert carries a reference and a proof, never the payload. Its validity does not depend on whether the accuser participated in the session, nor on the accuser's reputation or role.

D7 (Third-Party Event Verification, Root Attestation, and Equivocation Coverage). A third party c verifies a bundle $B = (A_s, e, R, \pi)$ in two parts.

Event verification:

1. $\text{anchor_ref} = H(\text{JCS}(A_s))$, and sigma_a and sigma_b verify under IKa and IKb ;
2. $e.\text{sender}$ agrees with d and sigma verifies under the session key that A_s binds to $e.\text{sender}$;
3. $e.\text{ts}$ lies inside $[T_{\text{start}}, T_{\text{end}}]$;
4. π is a valid inclusion proof for e under R ; and
5. R is signed by at least one participant for the same $(\text{sid}, d, \text{index range})$.

This satisfies the acceptance-side evidence requirement. A single-party attested root is sufficient to verify one event's inclusion.

Coverage checking:

- $\text{RootAttested}(R, \text{sid}, d, \text{range})$ holds when at least one participant signed R for $(\text{sid}, d, \text{range})$.
- $\text{EquivocationDetected}(\text{sid}, d, \text{range})$ holds when two attested roots R_1 and R_2 cover the same $(\text{sid}, d, \text{range})$ and $R_1 \neq R_2$. At least one participant signed an inconsistent root. The verifier records this fact and returns $\text{REJECT}(\text{EQUIVOCATION_DETECTED})$; it does not choose a preferred root.

Immediate third-party detection of truncation or deletion requires roots over the same range from multiple parties or other honest publishers. If an honest participant later publishes a root covering the truncated range and roots are published at interval r , detection occurs within at most r events. If no root ever covers that range, no finite delay bound applies. While no attested root covers the range, deletion or truncation inside it is invisible to third parties; the verifier returns $\text{UNCONFIRMED}(\text{ROOT_UNAVAILABLE})$ and retries rather than declaring the evidence invalid.

D7 therefore distinguishes:

- acceptance root: supports event inclusion for one alert; and
- coverage root: supports comparison over a range and detects truncation or equivocation.

D8 (Evidence Retrieval and Unavailability). A bundle $B = (A_s, e, R, \pi)$ can be served by the sender, receiver, or any peer that announced a matching root. The verifier trusts no provider and validates every candidate under D7.

1. The verifier derives candidates from the anchor, published roots, and requested (d, seq) .
2. It requests bundles with a deadline.
3. Each arriving bundle is checked for structure, anchor validity, signatures, and inclusion proof.
4. A malformed, unsigned, or invalid candidate is discarded. The provider is penalized locally. A single bad provider does not determine the final verdict.
5. If at least one valid bundle is obtained, D9 continues.
6. If no valid bundle is obtained before the TTL expires, the verifier returns $\text{UNCONFIRMED}(\text{EVIDENCE_UNAVAILABLE})$ and may retry.
7. After the TTL expires, it returns $\text{REJECT}(\text{STALE_ALERT})$.

Retrieval assumes at least one honest holder and at least one participant publishing roots, i.e., A6. Unavailability affects availability only; it never becomes acceptance.

D9 (Three-State Alert Acceptance). A verifier v returns:

$$\text{Decision}_v(a) \in \{\text{ACCEPT}, \text{REJECT}(\text{reason}), \text{UNCONFIRMED}(\text{retry_reason})\}.$$

ACCEPT requires all of the following:

1. the anchor named by `anchor_ref` verifies and `ts_a` lies inside its window and the alert TTL;
2. e verifies by D5 or D7, and $e.ts$ lies inside the anchor window and alert TTL;
3. `sigma_accuser` verifies and `copy_hash = H(JCS(e))`;
4. the referenced rule version remains resolvable; and
5. $r_{\{\text{rule_id}, \text{rule_version}\}}(e) = \text{violation}$.

Formally:

$$\begin{aligned} \text{Accept}_v(a) = 1 \iff & \text{AnchorOK}(A_s) \wedge \text{Fresh}(e, a, A_s) \wedge (D5(e) \vee D7(e)) \\ & \wedge \text{copy_hash} = H(\text{JCS}(e)) \wedge \text{Verify}_{IK_{acc}}(\sigma_{accuser}) = 1 \\ & \wedge r_{\text{rule_id}, \text{rule_version}}(e) = \text{violation}. \end{aligned} \quad (9)$$

REJECT means that at least one condition has been deterministically shown false, for example:

- the anchor exists but is invalid or expired;
- an event in a verified attested bundle has an invalid signature;
- the inclusion proof is definitively invalid;
- conflicting roots constitute equivocation;
- `copy_hash` does not match the verified event;
- the accuser signature is invalid;
- the rule version resolves but re-evaluation is not a violation;
- the alert or event is outside the TTL;
- a retraction is invalid.

UNCONFIRMED means that a public object needed to complete the decision is temporarily unavailable, while no security condition has been shown false, for example:

- the anchor cannot yet be resolved;
- an attested root is unavailable;
- a valid bundle is unavailable;
- the retrieval deadline has expired but the alert TTL has not.

Priority:

1. If any verified condition is deterministically false, return REJECT.

2. If all conditions hold, return ACCEPT.
3. Return UNCONFIRMED only when there is no deterministic failure and required input is temporarily unavailable.
4. UNCONFIRMED may be retried before the TTL. After the TTL, it becomes REJECT(STALE_ALERT).

D9 never means "cannot decide, therefore accept." It also never means "a malicious provider returned bad data, therefore reject." Only a deterministic failure on a verified object produces REJECT.

D10 (Retraction). A retraction is $ret = (retractor, ts, target, reason, sigma_retractor)$, where $target$ identifies an alert by $(accuser, anchor_ref, d, seq, rule_id, rule_version, copy_hash)$ and $sigma_retractor = Sign_IK_retractor(JCS \text{ of the preceding fields})$.

A verifier honors a retraction only if $retractor$ equals the accuser named in the targeted alert and a record of that alert exists locally. It marks the alert ALERT_RETRACTED while retaining the original alert, evidence, and verdict.

Retraction is a statement about the accusation, not about the evidence. It deletes nothing, cannot make an invalid alert valid, and cannot erase valid evidence. Another party that obtains the same bundle may issue the same alert afresh.

D11 (Accusation Eligibility). Any party holding a valid bundle may issue an alert. The protocol places no role restriction on who may accuse, and the D9 acceptance predicate is independent of the accuser's identity, reputation, role, and session participation.

A deployment may discount an accepted alert after acceptance or limit its response policy. It may not fold identity into the D9 acceptance predicate, because that reintroduces a trust anchor and lets a well-connected party suppress valid evidence by declaring the accuser ineligible.

3.5 Threat Surfaces and Boundaries

1. **Induced violations.** An adversary can make an honest agent commit a real violation and then accuse it correctly. The protocol verifies evidence; it does not determine whether the violation was induced. Mitigations include locality of any response, operator review, and auditing induced-violation patterns.
2. **Alert-layer denial of service.** Rejected alerts still consume verification and retrieval. The protocol prevents false acceptance, not resource cost. Deduplication, rate limiting, and cached typed rejections reduce but do not eliminate cost.
3. **Equivocation and root coverage.** If two attested roots over the same $(sid, d, range)$ differ, the mismatch is detected. If no root covers a range, truncation inside that range is invisible to third parties. If an honest participant later publishes a root covering the truncated range and roots are published at interval r , detection occurs within at most r events; if no root ever covers that range, no finite delay bound applies.
4. **Rule-version drift.** A verifier lacking the referenced version returns RULE_UNKNOWN. This is safe but reduces interoperability. A9 requires the version to remain resolvable for the evidence-retention horizon. Determinism holds across verifiers sharing a version, not across arbitrary deployments.
5. **Privacy.** In the tested construction, alerts do not carry designated payload markers, designated instruction-injection markers, or a designated executable marker. This is payload privacy, not anonymity. Alerts may expose sid , direction, sequence number, timing, and rule identifiers; root publication may expose session volume and interaction patterns. Salting handles, federation-scoped logs, and short-lived session identifiers reduce metadata exposure.

6. **Availability and liveness.** If every holder of a session’s evidence is dishonest or unreachable, an honest verifier returns UNCONFIRMED before the TTL and STALE_ALERT afterward. The failure mode is false rejection, never false acceptance, but it allows an adversary to suppress accusations. E5 measures this behavior.

4 Method

4.1 Task Formulation

Inputs: an A2A interaction sequence with session credentials, identity-binding material, a deployment rule set, rule-version content, session anchors, and evidence-publication material.

Outputs: a signed chain-linked AuditEvent per interaction; a RevokeAdvice per observed violation; and ACCEPT, REJECT, or UNCONFIRMED for each alert. Participants and non-participants evaluate the same D9 predicate.

Scope: in-session behavioral evidence, accusation issuance, third-party evidence verification, and alert adjudication.

Out of scope: identity issuance primitives, authorization-model design, content-level injection detection, quarantine, recovery, and response policy.

4.2 Overview

The method has four modules:

- M1 produces evidence.
- M2 turns a violation into an alert.
- M3 executes the three-state D9 decision.
- M4 supplies root publication, retrieval, coverage detection, and throughput mechanisms.

All four are additive A2A extensions. M1 and M2 add data objects; M2 and M3 add operations; none changes L3 bindings.

The sender gateway signs an AuditEvent under its TEE-bound session key. The receiver verifies credential, quote, signature, chain head, and policy before admission. Both sides publish signed Merkle roots. An observer that detects a violation issues a RevokeAdvice naming an anchor, event position, rule version, copy hash, and inclusion proof. Any peer executes the same D9 predicate.

Root publication has two roles. A single attested root supports one event’s inclusion proof. Multiple roots over the same range support truncation and equivocation detection. The two roles must not be conflated.

4.3 M1 AuditEvent Construction

M1 reuses existing identity issuance and adds no identity primitive. Before a session, the gateway obtains and verifies IdentityBinding(IKi, i) through AgentCard or a verifiable credential. It then generates session keys inside the TEE, exchanges quotes, and co-signs the session anchor.

Interface	Input	Output	Treatment
IdentityResolver	IKi, AgentCard/VC	IdentityBinding(IKi,i) or IKBindingInvalid	Reuse existing identity mechanism

Interface	Input	Output	Treatment
RuleResolver	rule_id, rule_version	rule predicate or RULE_UNKNOWN	Deployment rule-publication system
RootPublisher	sid, d, index range, root	signed root	Added in this paper
BundleStore	anchor_ref, d, seq	bundle or unavailable	Added in this paper

A rule-resolution failure returns RULE_UNKNOWN and never guesses. If the remote peer does not support the extension, the session degrades to plain A2A and the local gateway records reduced assurance.

Algorithm 1 evidence generation:

```

on session_start(peer):
  check IdentityBinding(IK, self)
  (sk, pk) <- KeyGen_TEE()
  q <- Quote_TEE(IK, pk, sid, window)
  send q; receive q'; verify q'{}
  A <- (sid, IK, pk, IK', pk',
  h0_dir, h0_rev, window)
  anchor_sig <- Sign(IK, JCS(A))
  send anchor_sig; receive anchor_sig'{}
  on send(m):
    e <- (sid, dir, seq++, now(), self, peer, meta(m), scope(m),
    H(payload(m)), chain_head[dir])
    e.sigma <- Sign(sk, JCS(e))
    chain_head[dir] <- H(JCS(e))
    send(m, e); store(e)
  on receive(m, e):
    require VerifyCredential(e.sid) and VerifyQuote(e.sid)
    require VerifySig(pk_of(e.sender), e.sigma)
    and e.sender agrees with e.d
    require e.prev_hash = chain_head[e.d]
    and e.seq not in seen(e.d)
    require Policy(e.intent_meta, e.perm_scope, e) = ok
    admit(m); store(e)
    chain_head[e.d] <- H(JCS(e))

```

M1 is deliberately minimal: verification is $O(1)$ per event, payload content is never required, the O1/O2 boundary is preserved, and the sender's own key remains sufficient for later attribution because no peer holds it.

4.4 M2 Accuse: RevokeAdvice Issuance

Evidence does not travel by itself. Forwarding raw payloads would leak sensitive content and propagate adversarial text into the receiver's context. An accusation references evidence rather than reproducing it.

When an observer determines $r(e)$ = violation for the named version, Algorithm 2 constructs the alert, binding the session anchor, direction, sequence number, rule identity and version, re-evaluation result, event-copy hash, and inclusion proof.

```

on detect(e, r):
  require Verify(e) and r(e) = violation
  R, pi <- root_and_proof(e) # attested per D7
  a <- (self, now(), H(JCS(A_s)), e.d, e.seq,
  r.id, r.version, violation, H(JCS(e)), ref(R), pi)
  a.sigma <- Sign(IK_self, JCS(a))
  broadcast(a)

```

The evaluated alert construction carries no designated business-payload marker, designated instruction-injection marker, or designated executable marker. E8 additionally checks the listed injectable field names; generic executable-content detection is outside scope. The versioned rule identifier lets a verifier reproduce the exact predicate the accuser claims to have applied.

Who may accuse: eligibility is not an acceptance condition. Any party holding a valid bundle may accuse. A deployment may decide after acceptance how much response weight to give an alert, but it may not put identity back into D9.

Retracting a mistaken alert: a false positive produces a well-formed accusation about an agent that did nothing wrong. Since acceptance never depended on identity or standing, the accuser’s remedy is to publish the signed retraction of D10. Verifiers mark the alert ALERT_RETRACTED while retaining the original alert and evidence. A retraction cannot undo an already executed response, so response policy is outside the scope of this paper and is left to deployment-specific response layers.

4.5 M3 VerifyAdvice: Three-State Acceptance

If acceptance depended on accuser identity, reputation, or alert count, Sybil identities would decide outcomes. D9 must therefore be a deterministic function of public objects and public rules.

Participants use a local admitted event as a fast path. Non-participants retrieve and verify a bundle under D7. Both paths invoke the same D9 predicate and produce the same three-state verdict.

```
on receive(a):
  A <- resolve_anchor(a.anchor_ref)
  if A is unresolved:
    return UNCONFIRMED(ANCHOR_UNRESOLVED, retryable=true)
  if not VerifyAnchor(A):
    return REJECT(ANCHOR_INVALID)
  if not inside_window(now(), A):
    return REJECT(ANCHOR_EXPIRED)
  if not inside_window(a.ts, A) or now() - a.ts >{ } ttl:
    return REJECT(STALE_ALERT)
  B <- local_bundle(a)
  if B is missing:
    B <- retrieve(A, a.d, a.seq)
  if B is missing:
    if now() - a.ts >{ } ttl: return REJECT(STALE_ALERT)
    return UNCONFIRMED(EVIDENCE_UNAVAILABLE, retryable=true)
  if not VerifyBundleShape(B):
    discard_candidate(B)
  if now() - a.ts >{ } ttl: return REJECT(STALE_ALERT)
  return UNCONFIRMED(EVIDENCE_UNAVAILABLE, retryable=true)
  if conflicting_root(A, a.d, a.seq):
    return REJECT(EQUIVOCATION_DETECTED)
  if not root_is_attested(B.root, A, a.d, a.seq):
    return UNCONFIRMED(ROOT_UNAVAILABLE, retryable=true)
  if not VerifyInclusionProof(B.event, B.proof, B.root):
    discard_candidate(B)
  return UNCONFIRMED(EVIDENCE_UNAVAILABLE, retryable=true)
  e <- B.event
  if not VerifySig(key_of(e.sender, A), e.sigma):
    discard_candidate(B)
  return UNCONFIRMED(EVIDENCE_UNAVAILABLE, retryable=true)
  if a.copy_hash != H(JCS(e)):
    return REJECT(EVIDENCE_MISMATCH)
  if not VerifySig(a.accuser, a.sigma):
    return REJECT(ACCUSER_SIG_INVALID)
  rule <- resolve_rule(a.rule_id, a.rule_version)
  if rule is missing:
    return REJECT(RULE_UNKNOWN)
  if run_rule(rule, e) != violation:
    return REJECT(RULE_NOT_VIOLATED)
  accept(a)
  record(a, B)
```

Two constraints are essential.

First, deterministic failure has priority over unavailability. Invalid anchors, copy-hash mismatch, invalid accuser signatures, non-violations, and TTL expiry are deterministic failures.

Second, an unverified bad candidate cannot force a final rejection. Evidence providers are untrusted. A malformed candidate is discarded and retrieval continues. Before the TTL, the verifier returns UNCONFIRMED rather than treating one malicious provider’s bad data as a reason to reject the accusation.

When sufficient inputs exist, participants and non-participants execute the same D9 conditions.

4.6 M4 Commitments and Retrieval

4.6.1 Root Publication and Coverage

Each participant maintains outbound and inbound chains per session and periodically publishes a signed Merkle root for each chain, tagged with the covered (sid, d, index range).

Roots have two functions:

- acceptance proof: one participant's signed root can support an inclusion proof; and
- coverage detection: two or more roots over the same range can be compared. A mismatch returns REJECT(EQUIVOCATION_DETECTED). A match improves truncation coverage. No coverage returns UNCONFIRMED(ROOT_UNAVAILABLE).

Thus, D7's "at least one participant attests a root" and Section 3.5's "both participants publish to detect truncation" do not conflict. The former is the minimum for one event's verification; the latter is the coverage required for immediate anti-equivocation and truncation detection.

4.6.2 Log and Gossip

Roots are announced through a transparency log or gossip channel. The log is untrusted for soundness. A log that omits, delays, or reorders roots can cause availability failures, not false acceptance. The log adds durability, auditability, and a place for monitors to compare roots over the same range.

4.6.3 Retrieval

A bundle $B = (A_s, e, R, \pi)$ may be held by the sender, receiver, or any peer that announced a matching root. The verifier derives candidates from the anchor and announced roots, requests bundles with a deadline, validates each under D7, and continues D9 when a valid bundle arrives.

Providers are untrusted. A bad candidate is discarded and its provider is penalized locally. Only a deterministic failure on a verified object returns REJECT. If all requests fail before the TTL, the verifier returns UNCONFIRMED(EVIDENCE_UNAVAILABLE); after the TTL, it returns STALE_ALERT.

4.7 Parameters, Error Codes, and Edge Cases

Parameter	Meaning	Range
Session window	$[T_{\text{start}}, T_{\text{end}}]$ bound by the anchor	hours to days
Alert TTL	Validity window of an alert	minutes to hours
Root interval r	Merkle root and coverage period	seconds to minutes
Retrieval deadline	Time budget per bundle request	1-5 s
Max clock skew δ	Tolerated skew between honest clocks	≤ 2 s (A7)
Evidence retention	Event body retention before garbage collection	$\geq$ anchor window + alert TTL; default 30 days
Root retention	Anchor and signed tree-head retention	$\geq$ evidence retention; default 1 year

Edge cases:

- Out-of-order delivery: buffer by seq and admit when the predecessor arrives. D5(c) checks the head rather than assuming order.
- Duplicate alerts: deduplicate by (accuser, anchor_ref, d, seq, rule_id, rule_version, copy_hash).
- Unresolved anchor: return UNCONFIRMED(ANCHOR_UNRESOLVED) and retry before the TTL.
- Unavailable root: return UNCONFIRMED(ROOT_UNAVAILABLE), not invalid evidence.
- Unavailable evidence: return UNCONFIRMED(EVIDENCE_UNAVAILABLE); after TTL, REJECT(STALE_ALERT).
- Malformed candidate: discard it and continue retrieval. An untrusted provider cannot determine the final verdict.
- Conflicting roots: return REJECT(EQUIVOCATION_DETECTED), with no preference.
- Session re-keying: rotation creates a new sid and anchor; chains never span a rotation.
- Missing rule version: return REJECT(RULE_UNKNOWN). A9 requires resolvability during evidence retention.
- Retention and garbage collection: keep an event body at least until its anchor window and alert TTL expire. Afterward retain anchors, signed roots, and rule versions so existence and consistency proofs remain checkable.
- Retraction: record ALERT_RETRACTED without deleting the original alert or evidence.
- Rule publication: a retired version remains resolvable for the evidence-retention horizon; otherwise alerts naming it degrade to RULE_UNKNOWN.

4.8 Canonicalization, Extension Declaration, and Interfaces

4.8.1 Canonical Serialization

All signatures and hashes are computed over canonical bytes. AuditEvent, RevokeAdvice, and SessionAnchor are mapped to JSON according to their field tables and canonicalized with RFC 8785 JCS [45]. Unset optional fields are omitted; explicitly set defaults are retained. $\text{sig} = \text{Sign}(\text{sk}, \text{JCS}(x))$ and $\text{copy_hash} = \text{H}(\text{JCS}(e))$. Implementations are checked against fixed test vectors because a canonicalization mismatch presents as a signature failure and silently splits the federation.

4.8.2 Extension Declaration and Negotiation

The extension is declared in the AgentCard capabilities.extensions array with required=false under:

`https://<federation-domain>/a2a/ext/evidence-backed-accusation/v1`

Peers negotiate at session establishment. If the remote side does not support the extension, the session degrades to plain A2A and the gateway records reduced assurance. Extension points are message metadata and task-status updates. L3 bindings are unchanged.

4.8.3 Identity-Binding Interface

This paper does not issue identities. It consumes IdentityBinding(IKi, i). If the binding is missing, expired, or unverifiable, the session cannot enter full-assurance mode. A0 is an external precondition for T2.

4.8.4 Rule-Resolution Interface

RuleResolver(rule_id, rule_version) returns a rule predicate or RULE_UNKNOWN. The verifier executes the rule locally and does not trust the result field in the alert. The version remains resolvable for the evidence-retention horizon (A9).

4.8.5 Session Establishment

Each party i generates (ski, s, pki, s) inside its TEE and obtains qi binding $(IK_i, pki, s, sid, T_{start}, T_{end})$. The parties set $sid = H(IK_a \parallel IK_b \parallel na \parallel nb)$ from fresh nonces, exchange and verify quotes and credentials, derive the two initial heads from the anchor fields, and co-sign A_s . If confidentiality between gateways is needed, they additionally derive K_{enc} by ephemeral Diffie-Hellman; event authentication never depends on K_{enc} . Rotation creates a new sid and anchor. Authentication, quote, or anchor-signature failure aborts the session with a typed error.

4.9 Consistency of Formal Artifacts

The formal artifacts share one state and error model. Appendix B defines the decision states ACCEPT, REJECT, and UNCONFIRMED; anchor errors ANCHOR_UNRESOLVED, ANCHOR_INVALID, and ANCHOR_EXPIRED; root errors ROOT_UNAVAILABLE and EQUIVOCATION_DETECTED; evidence errors EVIDENCE_UNAVAILABLE, EVIDENCE_MISMATCH, and PROOF_INVALID; signature errors EVIDENCE_INVALID, ACCUSER_SIG_INVALID, and RETRACTION_INVALID; rule errors RULE_UNKNOWN and RULE_NOT_VIOLATED; the time error STALE_ALERT; and the retraction state ALERT_RETRACTED.

Appendix D maps R1-R5 and N1-N6 to D9. The security analysis in Section 5 and the evaluation results in Section 6 use the same definitions. In particular, EVIDENCE_UNAVAILABLE is not a final rejection, a single root is not sufficient for truncation detection, and D9 contains no term for identity, reputation, or alert count.

5 Security Analysis

5.1 Security Claims, Trust Boundaries, and Assumptions

This paper proves five claims within the evidence-and-accusation layers:

ID	Claim	Core Question	Result
G1	Evidence integrity	Can insertion, modification, reordering, replay, or bounded truncation be detected?	T1a/T1b
G2	Attributability	Can an accepted event be attributed to the anchor-bound sender?	T2
G3	Acceptance soundness	Can unsupported or non-violating alerts be accepted?	T3
G4	Third-party verdict determinism	Do participants and non-participants reach the same verdict on the same inputs?	T4

ID	Claim	Core Question	Result
G5	Availability and cost boundary	How are unavailability and Sybil floods bounded?	P1, P2, PR1

Trust boundaries:

Component	Trust Requirement	Treatment
TEE and remote attestation	Deployment assumption, not evaluated	A3: quotes are unforgeable and session keys cannot be exported. A3 is assumed and is not experimentally validated in this paper.
Long-term identity key IKi	Identity binding required	A0: IKi is uniquely bound to agent i by the identity mechanism
Deployment rules	Correctness not required	A4: rules are deterministic and versioned; governance is deployment responsibility
Transparency log and gossip	Not trusted for soundness	Omission, delay, and reordering affect availability, not acceptance
Evidence holders	Not trusted	Every bundle is independently verified under D7
Verifier implementation	Correct implementation required	A5 and A8 require exact D9 and JCS behavior

Assumptions:

- A0 (identity binding): IKi is uniquely bound to agent i and honest verifiers can resolve the binding.
- A1 (signatures): Ed25519 is EUF-CMA secure.
- A2 (hashing): SHA-256 is collision resistant.
- A3 (TEE): quotes are unforgeable and session private keys never leave the TEE. A3 is assumed and is not experimentally validated in this paper; hardware-TEE benchmarking [46] does not constitute validation of A3.
- A4 (deterministic rules): rules are deterministic predicates uniquely identified by rule_id and rule_version.
- A5 (implementation correctness): honest verifiers implement D9 exactly and do not replace evidence with identity, reputation, or alert count.
- A6 (evidence availability): each session has at least one participant publishing roots and at least one reachable honest holder during retrieval.
- A7 (clock bound): clock skew is bounded by delta, and every TTL and window exceeds 2 delta.
- A8 (canonicalization consistency): all honest verifiers use the same JCS canonicalization and test vectors.
- A9 (rule availability): any rule version cited by an alert remains resolvable for the evidence-retention horizon.

The analysis does not assume honest logs, honest providers, a correct deployment policy, or an anonymous metadata channel.

5.2 Security Games, Relations, and Lemmas

The adversary controls a subset of agent semantics and the network under Dolev-Yao, may create Sybils, and may suppress evidence or roots. It cannot obtain long-term or TEE session private keys, forge quotes, alter gateway logic, or break Ed25519 and SHA-256.

$$\text{Adv}^{\text{EUF-CMA}}(\lambda) = \Pr[\text{Exp}^{\text{euf-cma}}(\lambda) = 1], \quad (\text{F1})$$

$$\text{Adv}^{\text{CR}}(\lambda) = \Pr[(m_0, m_1) \leftarrow \mathcal{A}(\lambda) : m_0 \neq m_1 \wedge H(m_0) = H(m_1)], \quad (\text{F2})$$

$$\Pr[\mathcal{A} \text{ forges a quote binding } (IK_i, pk_{i,s}, sid, T_{\text{start}}, T_{\text{end}})] \leq \text{negl}(\lambda), \quad (\text{F3})$$

A1 and A2 give $\text{Adv}^{\text{EUF-CMA}}(\lambda) \leq \text{negl}(\lambda)$ and $\text{Adv}^{\text{CR}}(\lambda) \leq \text{negl}(\lambda)$; A3 is (F3).

Ed25519 verification establishes that the signature authenticates the message under the public key; it does not require bitwise equality with a particular signing transcript.

Basic relations:

$$\text{Verify}_{pk}(\sigma, m) = 1 \implies \text{Binds}_{pk}(\sigma, m), \quad (\text{F4})$$

$$e_{d,1}.\text{prev} = h_d^0, \quad e_{d,i}.\text{prev} = H(e_{d,i-1}) \quad (i \geq 2), \quad (\text{F5})$$

$$v = H(e_{d,i}), \quad R(v) = H(H(e) \parallel R(v_L) \parallel R(v_R)), \quad \text{VerifyProof}(e, \pi, R) = 1 \iff \pi \text{ recomputes } R \text{ from } H(e), \quad (\text{F6})$$

$$\text{copy_hash} = H(\text{JCS}(e)), \quad \text{anchor_ref} = H(\text{JCS}(A_s)), \quad (\text{F7})$$

$$\begin{aligned} \text{Fresh}(e, a, A_s) \iff & |now - e.ts| \leq \delta \wedge T_{\text{start}} - \delta \leq e.ts \leq T_{\text{end}} + \delta \\ & \wedge 0 \leq now - a.ts \leq \text{TTL}, \end{aligned} \quad (\text{F8})$$

Lemma 1 (anchor binding): If A_s verifies under IK_a and IK_b , then the session public key used to attribute an event to party i is the key quoted by i 's TEE for s . Substituting another key breaks A1 or A3.

Lemma 2 (equivocation detection): If two attested roots over the same (sid, d, range) differ, at least one participant signed an inconsistent root. The mismatch is evidence of misbehavior. If no root over a range is published, truncation inside that range is invisible to third parties.

Lemma 3 (verification determinism): Under A4, A5, and A8, $\text{Accept}_v(a)$ is a pure function of anchor, bundle, rule version, clock window, and public parameters. It is independent of the verifier's identity, role, reputation, and session participation.

5.3 T1 Evidence Integrity

5.3.1 T1a Chain Integrity and Replay Resistance

Theorem 1a: Under A1, A2, and A3, an adversary holding neither $sk_{\text{sender},s}$ nor the sender's long-term identity key cannot modify, insert, reorder, or replay events of a direction's chain without detection by D5 or D7.

Proof:

- Modification or insertion requires a signature that verifies under $pk_{\text{sender},s}$ on a message the sender never signed, violating A1.
- Reordering fails D5(c), since $e.\text{prev_hash}$ must equal the stored head, and D3 fixes the first head to $h0_d$.

- Replay fails D5(d) because $e.seq$ is unused in d . Cross-session or cross-direction replay also fails sequence, anchor, and initial-head checks.
- Cross-session substitution or key replacement fails because the anchor binds pki,s to IK_i through the TEE quote.

Thus, except for deletion or truncation, the undetected integrity-violation probability is bounded by $Adv_{EUF-CMA}(\lambda) + Adv_{CR}(\lambda)$, with no log, accuser, or identity-count term.

5.3.2 T1b Truncation and Equivocation Detection

Theorem 1b: Under A2, if two parties publish roots over the same $(sid, d, range)$ and the roots differ, the mismatch is identified as equivocation. If an honest participant later publishes a root covering the truncated range and roots are published at interval r , detection occurs within at most r events; if no root ever covers that range, no finite delay bound applies.

Proof: A truncation causes the sender's outbound root and the receiver's inbound root over the same range to differ. Since both roots are signed, the mismatch cannot be dismissed as a proof failure (Lemma 2). Without coverage, a third party has no commitment to compare and must wait for a later root. If a later honest root covers the range, the delay is at most r events. If no root ever covers the range, the absence of a commitment means that no finite detection bound exists.

$$\Pr[\text{undetected integrity violation}] \leq Adv^{EUF-CMA}(\lambda) + Adv^{CR}(\lambda) + \Pr[\text{truncation range not covered by any attested root}]. \quad (F9)$$

The first two terms are cryptographic. The third is a liveness term governed by A6; it is bounded analytically and is not a separately implemented attack in E1.

5.4 T2 Attributability and Non-Repudiation

Theorem 2: Under A0, A1, A2, and A3, every event admitted by D5 or verified by D7 is attributable to the holder of the identity and TEE session key bound by the anchor. If $IK_e.sender$ is correctly bound to agent i , the event is attributable to i . A sender cannot repudiate an event it signed, and a peer cannot frame the sender.

Proof: A D7 bundle carries σ verifying under the session key bound to $IK_e.sender$; D5(b) imposes the same check for participants. By A3, the private key exists only in the sender's TEE. A valid signature under it can only be produced there, and a peer producing it would violate A1. A0 connects the TEE-bound key to the real-world identity.

$$\Pr[\text{an admitted event attributed to } i \text{ was not produced by } i\text{'s bound key}] \leq Adv^{EUF-CMA}(\lambda) + \Pr[i\text{'s TEE exports } sk_{i,s}]. \quad (F10)$$

There is no log, accuser, or identity-count term. Attribution depends on key binding, not on transcript operations or alert volume.

Remark: If participants shared one symmetric session key, either party could sign events naming the other as sender and all verifiers would accept them. Per-party session keys are therefore necessary for T2.

Boundary: Attribution fails if $sk_{i,s}$ is disclosed, exported by the TEE, physically extracted, or if A0 is false.

5.5 T3 Acceptance Soundness and Framing Resistance

Theorem 3: Under A1, A2, A4, A5, and A8, if an honest verifier returns ACCEPT for alert a , then the bundle is valid, the event signature and anchor agree, $copy_hash$ matches, the accuser signature verifies, the alert and event are fresh, and the referenced rule version re-evaluates to violation.

Proof: D9's ACCEPT is the conjunction of the five conditions. Any deterministic failure requires REJECT. Temporary unavailability without a deterministic failure requires UNCONFIRMED, not ACCEPT. Therefore, every ACCEPT satisfies all required conditions.

Corollary 3.1 (framing resistance): Fabricated, inserted, altered, mislabeled, expired, or non-violating alerts cannot be accepted by an honest verifier. The number of Sybil identities does not change the per-alert acceptance probability.

Proof sketch: Forged anchors or event signatures violate A1. Altered evidence or copy hashes violate A2. Mislabeled rules are detected by local rule re-evaluation. Expired alerts fail window and TTL checks. Missing rule versions return RULE_UNKNOWN under A4/A9. Invalid accuser signatures fail verification. Except for sigma_accuser validity, D9 contains no accuser-identity term. N Sybil identities can produce at most N failed attempts and do not convert an invalid alert into a valid one.

$$\Pr[\exists \text{ honest } v : \text{Accept}_v(a) = 1 \wedge \text{InvalidEvidence}(a)] \leq \text{Adv}^{\text{EUF-CMA}}(\lambda) + \text{Adv}^{\text{CR}}(\lambda), \quad (\text{F11})$$

$$\Pr[\exists k \leq q : \text{Accept}_v(a_k) \wedge \text{invalid}] \leq q (\text{Adv}^{\text{EUF-CMA}}(\lambda) + \text{Adv}^{\text{CR}}(\lambda)), \quad (\text{F12})$$

where $q \leq |S|R_{\max}T_{\max}$ is the number of attempts.

$$\Pr[\text{equivocation detected} \mid R_1 \neq R_2] = 1, \quad \begin{array}{l} \text{if a later honest root covers the range, the} \\ \text{truncation-detection delay is at most } r; \\ \text{otherwise no finite delay bound applies.} \end{array} \quad (\text{F13})$$

$$\begin{aligned} \text{Accept}_v(a) = 1 \iff & \text{AnchorOK}(A_s) \wedge \text{Fresh}(e, a, A_s) \wedge (D5(e) \vee D7(e)) \\ & \wedge \text{copy_hash} = H(\text{JCS}(e)) \wedge \text{Verify}_{IK_{acc}}(\sigma_{accuser}) = 1 \\ & \wedge r_{\text{rule_id, rule_version}}(e) = \text{violation}. \end{aligned} \quad (\text{F14})$$

The q term appears in the multiple-attempt cost, not in the single-alert acceptance probability. Sybils increase attempts and cost, not per-attempt success.

Boundary: Framing resistance does not prohibit false accusations or eliminate false rejections. Sybils can consume CPU and retrieval resources. The evaluation measures only the local object-processing boundary.

5.6 T4 Third-Party Verdict Determinism

Theorem 4: Under A4, A5, and A8, if a participant verifier and a non-participant verifier obtain the same anchor, bundle, alert, rule version, and valid clock input, they produce the same ACCEPT, REJECT, or UNCONFIRMED verdict. If a non-participant cannot obtain evidence or a rule version, it returns UNCONFIRMED or RULE_UNKNOWN and never converts unavailability into acceptance.

Proof: The participant path verifies an event through D5. The non-participant path verifies a bundle through D7. Both then invoke D9. D5 and D7 determine how evidence is obtained and verified; they do not alter D9's anchor, freshness, copy-hash, signature, or rule conditions. Under A4, A5, and A8, the verdict cannot depend on session role, reputation, alert count, or private state.

Acceptance has three states. ACCEPT means all D9 conditions hold. REJECT means at least one condition is deterministically false. UNCONFIRMED means required public input is temporarily unavailable, and the verifier must not treat it as evidence of falsity or as acceptance.

5.7 Availability and Privacy

Proposition 1 (availability safety): Evidence unreachability, holder refusal, log delay, or retrieval timeout cannot cause an honest verifier to ACCEPT an alert with invalid evidence.

Proof: Retrieval failure produces EVIDENCE_UNAVAILABLE or UNCONFIRMED. D9 does not skip anchor, hash, signature, or rule checks when evidence is missing. The only safe output without evidence is non-acceptance.

Proposition 2 (conditional liveness): If A6 and A9 hold, the alert is within TTL, and evidence holders eventually respond, an honest verifier with the referenced rule version eventually reaches ACCEPT or deterministic REJECT rather than remaining UNCONFIRMED forever.

Proof: A6 supplies a published root and a reachable honest holder. A9 supplies the rule version. The verifier can retry and complete D9 before TTL expiry.

$$\Pr[\text{retrieval succeeds}] = 1 - (1 - \alpha)^k \quad (\text{F15})$$

under independent holder responses. Correlated failure, organization-wide outage, and malicious holder behavior weaken this bound and are measured separately in E5.

The accusation-storm cost is bounded by (F16):

$$C \leq q(t_{\text{verify}} + t_{\text{proof}} + t_{\text{retrieve}} + s_{\text{alert}}), \quad q \leq |S|R_{\max}T. \quad (\text{F16})$$

Privacy boundary: in the tested construction, RevokeAdvice carries no designated business-payload marker, designated instruction-injection marker, or designated executable marker. E8 also checks the listed injectable field names; generic executable-content detection is outside scope. This is payload privacy, not anonymity. Metadata such as sid, direction, sequence number, timing, rule identifier, and possibly accuser identity may remain visible. Root publication may expose session volume and interaction patterns. Salting, federation-scoped logs, and short-lived session identifiers reduce metadata exposure but do not hide disputed interactions.

5.8 ProVerif Formal Verification

The ProVerif model encodes honest and dishonest gateways, a Dolev-Yao network, TEEs as private channels issuing one session key pair per party and session, co-signed anchors, per-direction private head cells, root publication with equivocation checks, two independent holders, local rule evaluation, and a retraction message that updates records but not the acceptance predicate. The Merkle tree is abstracted as an unforgeable proof term because its security reduces to A2. The private channel is a symbolic encoding of A3; it verifies protocol behavior under A3, not hardware quote generation or key isolation.

Positive queries:

```
(* R1 authentication and agreement: an accepted event was honestly sent *)
query inj-event(AcceptedEvent(sid, d, seq, sender))
==>{} inj-event(HonestSend(sid, d, seq, sender)).
(* R2 chain binding: an admitted event's predecessor was admitted *)
query ev:Admit(sid, d, seq, h) ==>{} ev:ChainLink(sid, d, seq, h).
(* R3 acceptance implies valid evidence and a genuine rule violation *)
query ev:AcceptAlert(sid, d, seq, rule)
==>{} ev:ValidEvent(sid, d, seq) && ev:RuleViolation(sid, d, seq, rule).
(* R4 non-repudiation: an accepted alert was issued by that accuser *)
query inj-event(AcceptAlert(accuser, sid, d, seq))
==>{} inj-event(Accuse(accuser, sid, d, seq)).
(* R5 sender binding: the third-party path checked the session key bound to the claimed sender *)
query inj-event(AcceptedEvent(sid, d, seq, sender))
==>{} inj-event(AnchorBinds(sid, d, sender)).
```

Negative models:

ID	Modification	Expected Failure
N1	Shared session key	R5 fails
N2	Copy-hash check removed	R3 fails
N3	Anchor window check removed	Freshness or expiry assertion fails
N4	Local rule re-evaluation removed	R3 fails
N5	Single-sided root publication only	Truncation and equivocation coverage degrades
N6	Accuser signature check removed	R4 fails

Every run must archive the ProVerif version, model file, query text, command, transcript, and counterexample. If removing a mechanism does not change the result, the model is under-covering that mechanism.

5.9 Mapping to Evaluation Results

Claim	Result	ProVerif	Experiment	Pass Criterion	Failure Condition
Consistency and interoperability	A4, A5, A8	R1-R5 model premises	E0	Expected canonical bytes, codes, and verdicts for all fixed vectors and primary code paths	Same vector or input yields a different result
Evidence integrity	T1a and conflicting-root portion of T1b	R1, R2	E1	All tampering detected; conflicting roots rejected	Any injected tampering or root conflict accepted
Attributability and non-repudiation	T2	R1, R5	E2	Legitimate events attributable; peer forgery fails; N1 fails	Peer forges a sender event or N1 passes R5
Acceptance soundness and framing resistance	T3	R3, R4	E3	Zero invalid acceptance; full valid control acceptance	Any invalid acceptance or systematic valid rejection
Third-party verdict determinism	T4	R5	E4	Participant and non-participant verdicts agree on identical input	Different verdicts or unavailability treated as acceptance
Availability	P1, P2	Not directly covered	E5	Only REJECT or UNCONFIRMED on unavailability; eventual verdict under A6/A9	Unavailability causes ACCEPT or permanent UNCONFIRMED

Claim	Result	ProVerif	Experiment	Pass Criterion	Failure Condition
Payload and object privacy	PR1	Not covered	E8	No designated payload marker, designated instruction-injection marker, designated executable marker, or listed injectable field name in alerts, roots, proofs, or bundles	Any such marker or listed injectable field name is observed

5.10 Summary

T1 and T2 use cryptographic reductions under A0-A3. T1 separates chain integrity from coverage-bounded truncation detection. T3 is a logical proof from the conjunction of D9 conditions, and T4 is a deterministic proof that participant and non-participant paths preserve the same three-state verdict on identical inputs. P1/P2 separate availability safety from conditional liveness. Privacy is an independent boundary. The analysis does not prove identity issuance, rule correctness, O3 detection, quarantine, recovery, response policy, TEE physical security, hardware attestation, hardware key confinement, remote attestation, or implementation conformance.

6 Evaluation Results

The evaluation is positioned as protocol-level evidence and accusation evaluation under deployment assumption A3.

This section reports the completed evaluation of the evidence-and-accountability layer introduced in this paper. It covers evidence generation, anchoring, root publication, alert construction, three-state verification, and evidence retrieval. It does not evaluate isolation, containment, recovery, or response policy; those mechanisms are outside the scope of this paper.

The experiments tested the four core security results from Section 5, T1-T4, together with the availability boundaries P1/P2 and the payload/object-privacy boundary PR1. Pass and failure conditions were fixed before execution and are retained in the artifact. A functioning implementation is not, by itself, evidence that a security claim holds.

6.1 Research Questions and Falsifiable Claims

The reported evaluation addresses six research questions:

RQ1. What local overhead do AuditEvent construction, session anchoring, Merkle-root publication, inclusion-proof processing, and alert verification introduce in the prototype?

RQ2. Do participants and non-participants produce identical ACCEPT, REJECT, and UNCONFIRMED verdicts for the same valid inputs, and do they deterministically reject invalid alerts?

RQ3. Without a TEE session private key, can an adversary insert, modify, reorder, replay, or roll back evidence, or publish conflicting roots, without detection?

- RQ4. Can Sybil identities, alert floods, or unavailable evidence holders cause an invalid alert to be accepted?
- RQ5. Do the ProVerif queries R1-R5 hold, and do the negative models N1-N6 fail exactly where the removed mechanism is required?
- RQ6. Does alert processing leak business payloads or inject adversarial text into a receiver’s context?

Claim	Result	Experiment	Primary pass criterion	Failure condition
Evidence cannot be silently altered	T1a and conflicting-root portion of T1b	E1	Every injected tampering class is detected	An insertion, modification, reorder, replay, rollback, or conflicting-root case is accepted
Behavior is attributable to the TEE-bound sender	T2	E2	Legitimate events are attributable and a peer cannot forge them	A peer forges a sender event or the shared-key negative model passes R5
An unsupported alert cannot be accepted	T3	E3	Zero invalid alerts are accepted and every valid control alert is accepted	Any invalid acceptance or systematic rejection of valid controls
Participants and non-participants reach the same verdict	T4	E4	The three-state verdict is identical for identical input	Different verdicts, or unavailability treated as acceptance
Unavailability causes rejection or non-confirmation, never acceptance	P1/P2	E5	Every retrieval failure returns a typed REJECT or UNCONFIRMED	Unavailability causes ACCEPT
Local overhead is measurable	Q1	E6	Latency, object size, and component CPU cost are reported	Raw local measurements or a material cost term are omitted
The formal model covers the message-layer claims	T1-T4	E7	R1-R5 hold and N1-N6 fail as predicted	A correct model fails or removing a mechanism leaves the result unchanged
Alerts do not disclose tested payload markers or listed injectable fields	PR1	E8	No designated payload marker, designated instruction-injection marker, designated executable marker, or listed injectable field name appears in alert, root, proof, or bundle objects	Any such marker or listed injectable field name is observed

6.2 Prototype, Environment, and Artifacts

The prototype implements M1-M4:

- software-simulated TEE quote and session-key interface; hardware attestation is out of scope;
- Ed25519 signatures and per-session signing keys;
- SHA-256 hash chains;
- RFC 8785 JCS canonicalization;
- in-memory chain, root, and bundle storage;
- one Merkle tree per direction and session;
- direct root announcement and bundle-provider interfaces;
- the participant fast path and the third-party retrieval path;
- rule-version resolution, deterministic rule re-evaluation, and typed rejection;
- signed retractions and duplicate-alert deduplication.

The artifact manifest records the operating system, processor architecture, Python version and executable, and SHA-256 digests of the source, workload, rule implementation, result, and document files. The third-party verifier runs in a separate process and has no access to participant session state. The prototype instantiates session-key and quote handling through a software-simulated attestation interface. Hardware TEE attestation is treated as deployment assumption A3 and is outside the evaluation scope. The artifact does not report hardware attestation latency, hardware-enforced key confinement, or production TEE behavior.

A minimal interoperability check exercises JCS and Ed25519 against independent library implementations. It does not implement D9, evidence retrieval, or a second full verifier. E0-E4 therefore compare the participant and third-party code paths of the primary implementation; they do not claim agreement between independent D9 implementations.

The evaluation uses an in-memory direct provider for roots and bundles. Network transfer, persistent storage, transparency-log operation, and gossip schedules are outside the evaluated scope and are left to future work.

6.3 Workloads and Rule Set

6.3.1 Workloads

ID	Name	Agents	Interactions or alerts	Main variables	Purpose
W1	Benign baseline	2, one session	10^4 interactions	No violations	Measure baseline overhead
W2	Mixed workload	2, one session	10^4 interactions	Violation rate p in {0.25, 0.5, 1.0}	Measure evidence and alert generation
W3	Alert flood	2, one session	10^2 - 10^5 alerts	Sybil fraction f in {0.1, 0.5, 0.9, 1.0}	Measure deduplication and local cost

These are single-session prototype workloads. The separate third-party determinism experiment uses a verifier with no session chain, no session private key, and no participant-local state; it may use only the anchor, attested roots, evidence bundle, direct provider, and public rules.

6.3.2 Rule Set

The evaluation uses ALLOW-1, SCOPE-1, AMOUNT-1, DEPTH-1, RATE-1, and CONTENT-1. Each rule has a fixed version string, threshold, and default value. The machine-readable rule implementation in the prototype source is fixed for all runs.

Rule ID	Version	Class	Predicate	Default
ALLOW-1	v1	O1	e.intent_meta.target is in the buyer allowlist	Fixed per run
SCOPE-1	v1	O1	e.perm_scope is within the scope delegated to e.sender	Attenuation only
AMOUNT-1	v1	O1	e.intent_meta.amount is at most the cap	cap = 10^4 units
DEPTH-1	v1	O1	e.intent_meta.delegation_depth is at most d_max	d_max = 3
RATE-1	v1	O1	Calls attributed to e.sender in a 60-second window are at most r_max	r_max = 100
CONTENT-1	v1	O2	Receiver-local content policy applied to the payload	Participant reference policy only

The rule set is intentionally small and fixed in the prototype source. The artifact manifest records the SHA-256 digest of that source. The claim under test is that acceptance is reproducible and resistant to framing, not that any particular policy is correct. A verifier that lacks the referenced rule version returns REJECT(RULE_UNKNOWN); it must not accept, substitute a default rule, or silently disagree with another verifier.

6.4 E0: Consistency, Interoperability, and Boundary Conditions

E0 does not test an attack. It establishes that all later experiments rest on the same canonical byte representation, the same error mapping, and the same verdict semantics.

E0 checks:

- JCS fixed test vectors;
- required and optional fields in AuditEvent, RevokeAdvice, and SessionAnchor;
- canonical bytes, signatures, and hashes matching fixed reference vectors and independent JCS/Ed25519 library checks;
- identical ACCEPT, REJECT, and UNCONFIRMED mappings;
- identical typed reason codes, including UNCONFIRMED(EVIDENCE_UNAVAILABLE), UNCONFIRMED(PROOF_UNAVAILABLE), and REJECT(STALE_ALERT);

- identical duplicate-alert deduplication;
- out-of-order delivery, buffering, and admission after a predecessor arrives;
- chain separation across session rotation;
- EVIDENCE_UNAVAILABLE after evidence-holder garbage collection;
- RULE_UNKNOWN when the referenced rule version is unavailable;
- acceptance, rejection, and non-confirmation at clock-window boundaries;
- retraction changing record state without deleting the original decision.

The pass criterion is that the reference vectors and the primary code paths produce the expected canonical bytes, signature-verification results, reason codes, and final verdicts for every vector. Any canonicalization difference is reported as a reproducibility failure, not as performance variance.

6.5 E1: Evidence Integrity

E1 tests T1a and the conflicting-root portion of T1b using the manipulation classes supported by the prototype. The report records the detection mechanism actually observed in each case; it does not claim that every alert-based case reaches the later chain-head or direction check.

Class	Injected manipulation	Observed detection mechanism
Modification	Change sender, seq, ts, payload_hash, or prev_hash	UNCONFIRMED(EVIDENCE_UNAVAILABLE) from event-signature or sequence/time consistency checks
Insertion	Insert an unsigned event or an event signed with another session key	UNCONFIRMED(EVIDENCE_UNAVAILABLE) from the D7 signature or anchor-binding check
Reordering	Supply a later genuine event for the alert's earlier position	UNCONFIRMED(EVIDENCE_UNAVAILABLE) because event signature, sequence, or proof/event binding fails before a chain-head comparison
Cross-direction replay	Move an a-to-b event into the b-to-a chain	UNCONFIRMED(EVIDENCE_UNAVAILABLE) from sender-key or direction consistency
Cross-session replay	Move an event from an old session into a new session	UNCONFIRMED(EVIDENCE_UNAVAILABLE) from sid, anchor, session key, or proof binding
Chain-head rollback	Restore an old chain head and append an event	UNCONFIRMED(CHAIN_HEAD_ROLLBACK) from the D5 replay and order check
Conflicting roots	Publish different roots for the same (sid, d, range)	REJECT(EQUIVOCATION_DETECTED)

Pass criteria:

- modification, insertion, reordering, cross-direction replay, cross-session replay, and chain-head rollback are detected in every injected case within the experimental boundary;
- conflicting roots are always recorded as equivocation evidence;
- every detection returns the correct typed decision and reason rather than a generic verification failure.

The result is limited to these injected classes. Truncation and multi-root coverage remain security-analysis properties and are not claimed as separately implemented E1 attacks.

6.6 E2: Attributability and Non-Repudiation

E2 tests T2 with three positive experiments and one negative control.

Positive experiments:

- The sender denies a genuine event that it signed. The verifier must attribute the event through the session public key bound by the anchor and TEE quote.
- The receiver attempts to construct an event in the sender's name. The attempt must fail because the receiver lacks the sender's TEE-protected session private key.
- An adversary replaces a session public key or reuses a key from another session. The attempt must violate Lemma 1 and return an anchor-binding or signature error.

Negative control:

- Modify the protocol so that both parties share one symmetric session key, then rerun R5 and the attribution experiment.

If the shared-key model still satisfies R5, the model or implementation does not verify that the sender possesses an independent private key. The negative model must fail, and its counterexample trace is reported as a formal result.

The pass criterion is that every legitimate event is attributable to the correct identity and session, neither the peer nor a third party can forge a sender event, and the shared-key model cannot satisfy R5.

6.7 E3: Acceptance Soundness and Framing Resistance

E3 tests T3 by breaking each acceptance condition in D9 separately. It is not sufficient to generate one generic "bad alert."

Attack or failure	Injection	Expected result
Forged anchor	Alter the signature, fields, or validity window	REJECT(ANCHOR_INVALID) or REJECT(ANCHOR_EXPIRED)
Forged evidence	Use an invalid event signature or a nonexistent reference	Discard the candidate; if no valid evidence remains, UNCONFIRMED(EVIDENCE_UNAVAILABLE), then REJECT(STALE_ALERT) after TTL expiry
Altered copy	Change the verified event content while preserving copy_hash	REJECT(EVIDENCE_MISMATCH)

Attack or failure	Injection	Expected result
Incorrect inclusion proof	Use a proof for another event or range	Discard the candidate; if no valid proof remains, UNCONFIRMED(PROOF_UNAVAILABLE), then REJECT(STALE_ALERT) after TTL expiry
Conflicting roots	Use two different roots for the same range	REJECT(EQUIVOCATION_DETECTED)
Forged accuser signature	Sign with a key not bound to accuser	REJECT(ACCUSER_SIG_INVALID)
Mislabeled rule	Claim violation while local re-evaluation returns ok	REJECT(RULE_NOT_VIOLATED)
Missing rule version	Reference an unpublished or retired version	REJECT(RULE_UNKNOWN)
Expired alert	Exceed TTL or the anchor window	REJECT(STALE_ALERT)
Future or misplaced time	Exceed the allowed clock skew	REJECT(TIME_WINDOW_INVALID)
Forged retraction	Sign a retraction with a key other than the original accuser's key	REJECT(RETRACTION_INVALID)

The experiment also includes a valid control set. Pass criteria:

- zero invalid alerts are accepted;
- every valid control alert is accepted;
- the final verifier state is only REJECT or UNCONFIRMED for invalid candidates;
- a deterministic failure on an already verified object produces REJECT, whereas a malformed candidate supplied by an untrusted provider produces UNCONFIRMED until the TTL expires;
- the distribution of rejection and non-confirmation reasons matches the injected failure class;
- increasing the Sybil fraction from $f = 0.1$ to $f = 1.0$ does not change the per-alert acceptance probability;
- increasing the number of Sybil identities changes attempts and cost, not the T3 false-acceptance bound.

A verifier that rejects every alert would also report zero invalid acceptances. The valid control set is therefore mandatory.

6.8 E4: Third-Party Verdict Determinism

E4 tests T4 by comparing the participant fast path with the non-participant retrieval path.

Experimental setup:

- the participant holds the local inbound chain;
- the non-participant holds no session state;
- both paths invoke the same D9 verifier implementation in separate processes;
- the non-participant resolves the anchor, discovers roots, retrieves evidence, verifies the inclusion proof, and re-evaluates the rule;

- both paths use the same rule version, evidence bundle, and clock window;
- the test covers complete evidence, missing evidence, a provider that returns incorrect evidence, a deterministic copy mismatch, and a forged anchor.

Pass criteria:

- for the same inputs, the participant and non-participant produce identical ACCEPT, REJECT, or UNCONFIRMED verdicts;
- when evidence is unavailable, both return UNCONFIRMED and neither returns ACCEPT;
- unavailability does not alter the hash, signature, rule, or time conditions in D9;
- the comparison does not claim agreement between independent D9 implementations.

The result is reported as the complete verdict matrix, not merely as a statement that a third party can accept a valid alert.

6.9 E5: Availability and Liveness

E5 tests P1/P2 under at least the following failure modes:

- each independent holder refuses to respond with probability α ;
- an organization or all holders are simultaneously offline;
- an evidence provider returns incorrect, stale, or forged evidence;
- evidence becomes available before TTL expiry and, separately, after TTL expiry.

Pass criteria:

- every retrieval failure produces EVIDENCE_UNAVAILABLE or UNCONFIRMED, or REJECT(STALE_ALERT) after TTL expiry;
- no unavailability condition causes an invalid acceptance;
- independent-holder retrieval success follows $1 - (1 - \alpha)^k$;
- correlated failures are reported separately and are not mixed into the independent-failure curve;
- when A6 and A9 hold and TTL has not expired, a valid alert eventually reaches ACCEPT or a deterministic REJECT rather than remaining UNCONFIRMED forever.

If unavailability causes acceptance, the result refutes P1. If a valid alert under the stated availability assumptions can never be confirmed, the result limits P2 rather than T3.

6.10 E6: Performance and Local Overhead

6.10.1 Microbenchmarks

Microbenchmarks measure these components separately:

- software-simulated TEE interface; hardware attestation excluded;
- AuditEvent construction, JCS canonicalization, hashing, and signing;
- event verification;
- Merkle leaf hashing, internal-node hashing, and root computation;
- inclusion-proof generation and verification;
- anchor resolution and verification;
- object serialization, deserialization, and local direct-provider lookup;
- rule re-evaluation;
- single-signature verification.

Microbenchmarks use 10^5 iterations. The report records warm-up policy and measurement boundaries.

6.10.2 End-to-End Experiments

End-to-end experiments measure:

- evidence-generation latency t_{ev} ;
- participant alert-verification latency t_{av} ;
- root-generation time and serialized bytes per interval;
- serialized object size for AuditEvent, RevokeAdvice, roots, and proofs;
- local CPU cost and deduplication under an alert flood.

Latency is reported as p50, p95, and p99. Means, standard deviations, medians, and interquartile ranges are selected according to the metric. The report separates measured local object processing from network, persistent-storage, and deployment-scale effects, which are not tested in E6.

6.11 E7: ProVerif Positive and Negative Verification

E7 covers T1-T4. The positive queries are R1-R5. E7 verifies message-layer semantics under A3 and does not validate hardware attestation.

- R1: authentication and agreement;
- R2: chain binding;
- R3: acceptance implies valid evidence and a genuine rule violation;
- R4: non-repudiation;
- R5: sender binding on the third-party path.

The negative models are:

ID	Modification	Expected failure
N1	Shared session key	R5 fails
N2	Remove copy_hash comparison	R3 fails
N3	Remove anchor-window checking	Freshness or expired-alert assertion fails
N4	Remove local rule re-evaluation	R3 fails
N5	Permit single-sided root publication only	Truncation and equivocation coverage degrades
N6	Remove accuser-signature verification	R4 fails

The pass criterion is that every positive query holds, every negative model fails as predicted, and each negative model produces a counterexample trace. If removing a mechanism leaves every query unchanged, the model under-covers that mechanism; the result does not establish redundancy. The artifact archives the ProVerif version, model file, query text, command, execution log, and counterexample.

6.12 E8: Payload and Object Privacy

E8 tests PR1:

- whether RevokeAdvice, root, proof, or retrieved-bundle objects contain a designated business-payload marker;
- whether those objects contain a designated instruction-injection marker that could inject adversarial text into a receiver's context;
- whether those objects contain a designated executable marker;
- whether any scanned object contains one of the listed injectable field names.

Pass criteria:

- no designated business-payload marker appears in alert, root, proof, or bundle objects;
- no designated instruction-injection marker, designated executable marker, or listed injectable field name appears in any scanned object;
- the result is described as payload/object privacy and never as anonymity or unlinkability.

No designated payload marker, designated instruction-injection marker, designated executable marker, or listed injectable field name was observed. Generic executable-content detection is outside scope.

E8 does not test transparency-log, gossip, network-metadata, or interaction-graph privacy.

6.13 Statistics, Reproducibility, and Validity Threats

E4 ran each scenario 30 times. In the full profile, E1 and E3 ran approximately 1,000 trials per attack class. E6 used 10^5 microbenchmark iterations, W1/W2 used 10^4 interactions, and W3 used 10^5 alerts per Sybil fraction. The protocol uses deterministic seeds and predeclared exclusion rules. Latency is reported as p50/p95/p99. Overhead is reported as mean and standard deviation or median and interquartile range. Attack experiments

report invalid-acceptance count, decision and reason classes, and detection rate. No bootstrap confidence interval or paired-comparison claim is made.

Raw outputs, rule source, JCS vectors, models, and counterexample traces are archived after each run. Failed runs are not deleted afterward. A zero-invalid-acceptance result is reported with its sample size and with the acceptance rate of the valid control set, so that "reject everything" cannot be mistaken for soundness.

Validity threats include:

- Hardware TEE attestation and key confinement are outside evaluation scope; simulations do not validate A3.
- synthetic workloads do not fully represent a production federation;
- the rule set is representative rather than exhaustive;
- JCS and error-code differences across implementations can cause cross-vendor failures;
- a finite experiment supplies an empirical detection count, not a proof that an attack is impossible in deployment.

Pass criteria, failure conditions, sample sizes, random seeds, and exclusion rules are fixed before execution. Any post-hoc change is reported as a new experiment version and does not overwrite the original result.

7 Discussion

Competing explanations. Centralized revocation is insufficient. Standard CRL [5] and OCSP [6] mechanisms express loss of authorization. A hijacked agent's credentials remain valid, and federations do not share one revocation authority. A credential-level mechanism answers whether a credential is still trusted, not what an agent did on a particular task. Reputation is not a substitute for evidence: reputation signals for language-model agents lack a stable identity grounding [7]. D9 contains no reputation term, so a Sybil set can affect an outcome only by producing evidence, never by increasing its cardinality. Improved detection is orthogonal. This paper defines what happens after a violation has been observed and enables the output of any detector to be verified by parties that trust neither the detector nor the accuser.

Deployment. M1-M3 are additive: a new L1 object, a new L2 operation, and no mandatory L3 change. A deployment may enable them per peer pair or per federation. Rules and thresholds are configuration, not protocol constants. Because alerts carry references and proofs rather than payloads, the design fits privacy constraints that prohibit forwarding business content. When a public log is unacceptable, root publication can point to a federation-internal log, and salted handles can keep the interaction graph out of that log. The requirement that cannot be removed is A6: a federation that publishes no roots retains attribution but loses third-party re-verifiability. A9 is the corresponding requirement for rule-version resolution. A deployment may run with software keys, but cannot claim hardware-backed attribution without a real TEE.

Claim-specific limitations. (i) The layer covers O1/O2 violations. An injected instruction that leaves no machine-readable trace remains outside its scope. (ii) Rule false positives create well-formed but incorrect accusations. The design requires determinism, not policy correctness; rule governance, version management, and review remain deployment responsibilities. (iii) Acceptance requires availability, so an adversary can suppress an accusation by suppressing evidence, although it cannot forge one. E5 tests the safety boundary, and E6 measures local object-processing cost; network retrieval and storage cost are outside the evaluated scope. (iv) Equivocation and truncation detection depends on root coverage. Detection is immediate when roots cover the same range. If an honest participant later publishes a root covering the truncated range and roots are published at interval r , detection occurs within at most r events; if no root ever covers that range, no finite delay bound applies. (v) Logs are untrusted for soundness. They cannot cause false acceptance, but an equivocating log can hide evidence from an auditor and delay detection. (vi) A signed retraction (D10) can withdraw an incorrect but well-formed accusation, but the retraction is advisory: a verifier retains the original

decision. A deployment that reacts immediately when an alert is accepted will still react to a false positive before the retraction arrives. Containment and response policy are therefore outside the scope of this paper; evaluating them together with this layer is left to future work. (vii) The privacy guarantee covers payloads, not the existence or timing of an accusation. An observer may learn that a particular interaction is disputed, and metadata such as sid, direction, sequence number, timing, and possibly accuser identity may remain visible. (viii) A3 is a deployment assumption; this paper does not validate quote unforgeability, hardware key confinement, or measured code.

8 Conclusion

This paper defines evidence-backed accusation for A2A: two additive extension objects, a three-state acceptance rule that a non-participant can execute, four core security results, a ProVerif model with R1-R5 and N1-N6, and a completed evaluation that separates soundness from availability. T1 and T2 use cryptographic reductions; T3 and T4 use logical and deterministic proofs. The claim is narrow and testable within its boundary: an accusation is credible exactly when public evidence and public rules support it, while the accuser’s identity affects the verdict only through the validity of its signature. This removes reputation and Sybil pressure from the acceptance decision and supplies the trusted input required by a distributed response layer.

Two smaller objects complete the layer on their own terms. A signed retraction allows an incorrect accusation to be withdrawn without erasing the record, and an explicit position holds that eligibility is a policy rather than a protocol condition, so status can never substitute for evidence. Section 6 reports completed experiments with predeclared pass and failure criteria that make each claim clearly falsifiable. Isolation, quarantine, recovery, and response policy are deliberately left outside the scope of this paper. TEE attestation remains deployment assumption A3 and is outside the evaluated scope.

9 Declarations

Competing interests. The authors declare no competing interests.

Funding. To be completed by the authors: funding agency and grant number.

Data availability. The prototype, ProVerif model, published rule set, canonicalization test vectors, workload generator, and raw measurement outputs described in Appendix E are archived as one artifact and will be released with the paper. The A2A extension objects, acceptance rule, and state/reason codes are fully specified in Section 4.8 and Appendix B, so the protocol can be reimplemented from the text alone.

Author contributions. To be completed by the authors: conceptualization, methodology, software, validation, formal analysis, and writing.

A Notation

Table A1. Notation.

Symbol	Meaning
sid, d, seq, ts	Session identifier, direction, sequence number, and timestamp
IK _i , sk _{i,s} , pk _{i,s}	Long-term identity key of i; session signing private/public key of i in session s
q _i	TEE quote binding (IK _i , pk _{i,s} , sid, T _{start} , T _{end})
A _s , anchor _{ref}	Session anchor and its canonical hash
h0 _d	Initial chain head for direction d

Symbol	Meaning
e, R, pi, root_ref	AuditEvent, attested Merkle root, inclusion proof, and root reference
copy_hash	H(JCS(e)) for the referenced event
O1/O2/O3	Metadata-visible, receiver-visible, and content-invisible violations
r, rule_id, rule_version	Deterministic rule predicate and its versioned identifier
ret	Signed retraction and target tuple (D10)
alpha	Per-holder response probability in the availability relation
D5/D7/D8/D9	In-session admission, third-party verification, retrieval, and three-state acceptance
A0	Identity-binding assumption: IK _i is uniquely bound to agent identity i
A9	Rule-version resolvability assumption for the evidence-retention horizon
ACCEPT, REJECT, UNCONFIRMED	Three-state verifier outcomes
E0-E8	Evaluation plan: consistency, integrity, attribution, soundness, determinism, availability, local performance, ProVerif, and payload/object privacy
Adv_EUF-CMA, Adv_CR	Advantages against signature unforgeability and hash collision resistance

B Object Schemas and State/Reason Semantics

All signatures and hashes are computed over JCS canonical bytes. Unset optional fields are omitted. Explicitly present defaults remain present. anchor_ref = H(JCS(SessionAnchor)), root_ref = H(JCS(RootAttestation)), and copy_hash = H(JCS(AuditEvent)).

```

AuditEvent {
  "sid": string,
  "d": "a-> b" | "b-> a",
  "seq": uint64,
  "ts": int64,
  "sender": string,
  "receiver": string,
  "intent_meta": object,
  "perm_scope": object,
  "payload_hash": hex32,
  "prev_hash": hex32,
  "sig": hex64
}
SessionAnchor {
  "sid": string,
  "ik_a": hex32,
  "pk_a": hex32,
  "q_a": bytes,
  "ik_b": hex32,
  "pk_b": hex32,
  "q_b": bytes,
  "h0_ab": hex32,
  "h0_ba": hex32,
  "t_start": int64,
  "t_end": int64,
  "sig_a": hex64,
  "sig_b": hex64
}
RootAttestation {
  "sid": string,

```

```

"d": "a-> b" | "b-> a",
"start_seq": uint64,
"end_seq": uint64,
"tree_size": uint64,
"root": hex32,
"publisher": string,
"sig": hex64
}
RevokeAdvice {
"accuser": string,
"ts": int64,
"anchor_ref": hex32,
"d": "a-> b" | "b-> a",
"seq": uint64,
"rule_id": string,
"rule_version": string,
"result": "violation",
"copy_hash": hex32,
"root_ref": hex32,
"inclusion_proof": [hex32],
"sig": hex64
}
EvidenceBundle {
"anchor": SessionAnchor,
"event": AuditEvent,
"root_attestation": RootAttestation,
"inclusion_proof": [hex32]
}
Retraction {
"retractor": string,
"ts": int64,
"target": {
"accuser": string,
"anchor_ref": hex32,
"d": "a-> b" | "b-> a",
"seq": uint64,
"rule_id": string,
"rule_version": string,
"copy_hash": hex32
},
"reason": string,
"sig": hex64
}

```

Table B1. Non-rejecting outcomes and temporary states.

Outcome	State/reason code	Meaning	Retry
ACCEPT	ACCEPT	Every D9 condition holds for the anchored, verified evidence and rule version	No further verification needed
UNCONFIRMED	ANCHOR_UNRESOLVED	The anchor cannot yet be resolved or retrieved before TTL expiry	Yes, before TTL
UNCONFIRMED	ROOT_UNAVAILABLE	No attested root covers the event position or required comparison range	Yes, before TTL
UNCONFIRMED	EVIDENCE_UNAVAILABLE	No valid evidence bundle was obtained before TTL expiry	Yes, before TTL
UNCONFIRMED	PROOF_UNAVAILABLE	No valid inclusion proof was supplied by any untrusted provider before TTL expiry	Yes, before TTL

Outcome	State/reason code	Meaning	Retry
REJECT	STALE_ALERT	A prior UNCONFIRMED state reaches TTL without sufficient input	No; final

Table B2. Deterministic rejection codes.

Reason code	Deterministic condition	Retry
ANCHOR_INVALID	The anchor signature or required anchor fields are invalid	No
ANCHOR_EXPIRED	The anchor is outside its validity window	No
EVIDENCE_INVALID	A verified event has an invalid signature or invalid chain fields	No
EVIDENCE_MISMATCH	copy_hash differs from H(JCS(e)) for the verified event	No
PROOF_INVALID	A verified inclusion proof is definitively invalid for the attested root and event	No
EQUIVOCATION_DETECTED	Two attested roots over the same (sid, d, range) conflict	No; the mismatch is itself evidence
ACCUSER_SIG_INVALID	The accuser signature is invalid	No
RULE_NOT_VIOLATED	The referenced rule resolves but local re-evaluation is not violation	No
RULE_UNKNOWN	The referenced rule version is unavailable	No; final and non-retryable
TIME_WINDOW_INVALID	The event or alert violates the allowed clock window	No
RETRACTION_INVALID	The retraction is not signed by the original accuser or does not match a recorded alert	No

ALERT_RETRACTED is a record state, not a reason to rewrite history. A verifier that already reached a verdict retains the original alert, evidence, and decision, and marks the record as retracted.

A malformed candidate from an untrusted provider is discarded. It does not itself produce REJECT. Only a deterministic failure on an already verified object produces REJECT.

C Additional Formulas and Derivations

The overhead relations referenced in Section 5.7 consist of the following terms.

$$B_{ev} = |sid| + |d| + |seq| + |ts| \quad (10)$$

$$+ 2|id| + |intent| + |scope| + 2|h| + |\sigma|, \quad (F19)$$

$$B_{adv} = |accuser| + |ts| + |anchor_ref| + |d| + |seq| + |rule_id| + |rule_version| + |copy_hash| + |root_ref| + |\pi| = \lceil \log_2 n \rceil \cdot 32 \text{ bytes}, \quad (F20)$$

$$L_{msg} = t_{sig} + t_{verify} + t_{hash} + t_{policy} + t_{store}, \quad (F21)$$

$$L_{alert} = t_{anchor} + t_{retrieve} + t_{proof} + t_{verify} + t_{rule}, \quad (F22)$$

$$T_{quote_sim, amortized} = \frac{t_{quote_sim}}{m}, \quad m \geq 1, \quad (F23)$$

$$\rho = \frac{\sum B_{payload}}{\sum B_{ev}},$$

$$\text{serialized overhead} = B_{adv}/B_{payload} + B_{root}/B_{payload},$$

$$B_{root}/r = (|h| + |\sigma| + |sd|)/r. \quad (F24)$$

In (F20), $|\pi| = \lceil \log_2 n \rceil \cdot 32$ bytes for a chain containing n events. In (F23), m is the number of messages in the session. The root-publication term is amortized over r events. Here t_{quote_sim} denotes the software-simulated attestation interface, not hardware TEE latency.

The inclusion-proof term $|\pi|$ is logarithmic in chain length and is not dominant in B_{ev} or B_{adv} . The root-publication term is amortized over r events; that amortization is what makes anti-equivocation practical at short root intervals.

D ProVerif Model-to-Protocol Mapping

Table D1. Protocol steps and formal queries.

Protocol step	Model element	Query or check
Establish a TEE-bound session key pair	Private channels issue one key pair per party and session; the private half is never exported	R1 and R5 premises
Co-sign the anchor	A term carrying both identity keys, both session public keys, both initial chain heads, and two signatures	R1, R5
Sign an AuditEvent	A canonical term containing the predecessor head is signed under the sender's session private key	R1, R2
Advance the chain	One private state cell per direction, replaced only on admission	R2
Publish a root	A signed tree-head term announced on the public channel; roots over the same range are compared	R2 and the equivocation branch
Issue an alert	A signature over (anchor_ref, d, seq, rule_id, rule_version, copy_hash, root_ref, proof)	R3, R4
Verify acceptance	Match the five D9 conditions for anchor, evidence, proof, rule, and freshness	R3, R4

Protocol step	Model element	Query or check
Bind the sender	Look up the claimed sender’s key in the co-signed anchor	R5
Re-evaluate the rule	Run the deterministic rule evaluator locally on the retrieved event	R3
Retract an alert	Apply a signed retraction message to the verifier’s record without deleting the prior verdict	Not directly queried

The private channel is a ProVerif encoding of A3, not hardware validation.

Table D2. Negative models.

ID	Removed or weakened mechanism	Expected query failure
N1	Shared session key	R5
N2	copy_hash comparison	R3
N3	Anchor-window check	Freshness or expiry assertion
N4	Local rule re-evaluation	R3
N5	Single-sided root publication only	Truncation and equivocation coverage
N6	Accuser-signature verification	R4

E Reproducibility Checklist

Dependency declarations, seed policy, JCS consistency results, raw measurements, rule implementation, and ProVerif transcripts are archived in the artifact repository. The repository contains the workload generator, direct retrieval and root-generation code, the ProVerif model with separate R1-R5 query files, and the canonicalization test vectors. The manifest records the operating system, processor architecture, Python version and executable, and SHA-256 digests of the source, workload, rule implementation, result, and document files.

The artifact also records the full negative-model counterexample for N1-N6, the verdict matrix for E0-E4, the retrieval failure taxonomy for E5, the local latency and serialized-size data for E6, the ProVerif command line for E7, and the object-dump analysis for E8. Any failed run remains in the repository. Post-hoc changes create a new experiment version rather than overwriting the original result.

References

- [1] Agent2Agent (A2A) Protocol Specification, Version 1.0.0. A2A Project / Linux Foundation, 2025. <https://a2a-protocol.org/v1.0.0/specification> (accessed 2026-09-19).
- [2] E. Rescorla. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446, IETF, 2018. DOI 10.17487/RFC8446.
- [3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz. Not What You’ ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec), pp. 79-90, 2023. DOI 10.1145/3605764.3623985.

- [4] D. Gurevin, C. Jin, P. H. Nguyen, O. Khan, M. van Dijk. Secure Remote Attestation With Strong Key Insulation Guarantees. *IEEE Transactions on Computers*, 74(3):848-859, 2025. DOI 10.1109/TC.2023.3290870.
- [5] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, W. Polk. Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile. RFC 5280, IETF, 2008. DOI 10.17487/RFC5280.
- [6] S. Santesson, M. Myers, R. Ankney, A. Malpani, S. Galperin, C. Adams. X.509 Internet Public Key Infrastructure Online Certificate Status Protocol - OCSP. RFC 6960, IETF, 2013. DOI 10.17487/RFC6960.
- [7] B. A. Hu, H. Rong, M. Van Kleek. Dissociative Identity: Language Model Agents Lack Grounding for Reputation Mechanisms. In *FAccT '26: The 2026 ACM Conference on Fairness, Accountability, and Transparency*, 2026. DOI 10.1145/3805689.3806748.
- [8] A. Grigor, C. Schroeder de Witt, S. Birnbach, I. Martinovic. VET Your Agent: Towards Host-Independent Autonomy via Verifiable Execution Traces. In *Proceedings of the ACM Asia Conference on Computer and Communications Security (ASIACCS)*, pp. 820-834, 2026. DOI 10.1145/3779208.3786259; arXiv:2512.15892.
- [9] R. Xing et al. VCT: A Verifiable Transcript System for LLM Conversations. arXiv:2606.23003, 2026.
- [10] S. Haber, W. S. Stornetta. How to Time-Stamp a Digital Document. *Journal of Cryptology*, 3(2):99-111, 1991. DOI 10.1007/BF00196791.
- [11] R. C. Merkle. A Certified Digital Signature. In *Advances in Cryptology - CRYPTO '89*, LNCS 435, pp. 218-238. Springer, 1989. DOI 10.1007/0-387-34805-0_21.
- [12] J. He, D. Yu. Agent-Native Telemetry: Verifiable State-Delta Evidence for Autonomous Operations. arXiv:2608.16178, 2026.
- [13] Z. Zhou. Governing Dynamic Capabilities: Cryptographic Binding and Reproducibility Verification for AI Agent Tool Use (A2Auth). arXiv:2603.14332, 2026.
- [14] S. Jamshidi et al. Verifiable Manifest Signing and Transparency Enforcement for Secure MCP-Based LLM Pipelines. arXiv:2601.23132, 2026.
- [15] J. R. Douceur. The Sybil Attack. In *Proceedings of the First International Workshop on Peer-to-Peer Systems (IPTPS)*, LNCS 2429, pp. 251-260. Springer, 2002. DOI 10.1007/3-540-45748-8_24.
- [16] R. Kang, Y. Diponegoro. Governance Gaps in Agent Interoperability Protocols: What MCP, A2A, and ACP Cannot Express. arXiv:2606.31498, 2026.
- [17] C. Schroeder de Witt et al. Open Challenges in Multi-Agent Security: Towards Secure Systems of Interacting AI Agents. arXiv:2505.02077, 2025.
- [18] A2ABreak: Systematic Security Analysis of the A2A Protocol. ACSAC 2026. arXiv:2609.10871.
- [19] Y. Louck, A. Stulman, A. Dvir. Improving Google A2A Protocol: Protecting Sensitive Data and Mitigating Unintended Harms in Multi-Agent Systems. *ACM Transactions on Software Engineering and Methodology*, 2026. DOI 10.1145/3821216; arXiv:2505.12490.
- [20] Y. Louck, A. Stulman, A. Dvir. Security Analysis of Agentic AI Communication Protocols: A Comparative Evaluation. arXiv:2511.03841, 2025.
- [21] N. K. Krishnan. Beyond Context Sharing: A Unified Agent Communication Protocol (ACP) for Secure, Federated, and Autonomous Agent-to-Agent Orchestration. arXiv:2602.15055, 2026.
- [22] L. Liu, H. Yu. Attesting Outputs and Delegation Ancestry in Multi-Agent AI Systems. arXiv:2608.30387, 2026.

- [23] G. Guardiola-Muzquiz, E. Soriano-Salvador. SealFSv2: Combining Storage-Based and Ratcheting for Tamper-Evident Logging. *International Journal of Information Security*, 22(2):447-466, 2023. DOI 10.1007/s10207-022-00643-1.
- [24] B. Schneier, J. Kelsey. Secure Audit Logs to Support Computer Forensics. *ACM Transactions on Information and System Security*, 2(2):159-176, 1999. DOI 10.1145/317087.317089.
- [25] B. Laurie, A. Langley, E. Kasper. Certificate Transparency. RFC 6962, IETF, 2013. DOI 10.17487/RFC6962.
- [26] J. Zhou, D. Gollmann. A Fair Non-repudiation Protocol. In *Proceedings of the 1996 IEEE Symposium on Security and Privacy (S&P)*, pp. 55-61, 1996. DOI 10.1109/SECPRI.1996.502669.
- [27] A. Haeberlen, P. Kouznetsov, P. Druschel. PeerReview: Practical Accountability for Distributed Systems. In *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP)*, pp. 175-188, 2007. DOI 10.1145/1294261.1294279.
- [28] Z. Zou et al. InterSAGE: The Secure and Verifiable Interoperability Protocol for An Internet of Agents. *arXiv:2608.13030*, 2026.
- [29] S. Prakash. AIP: Agent Identity Protocol for Verifiable Delegation Across MCP and A2A. *arXiv:2603.24775*, 2026.
- [30] S. Prakash. LDP: An Identity-Aware Protocol for Multi-Agent LLM Systems. *arXiv:2603.08852*, 2026.
- [31] Z. Zou et al. BlockA2A: Towards Secure and Verifiable Agent-to-Agent Interoperability. *arXiv:2508.01332*, 2025.
- [32] S. Malkapuram et al. Context Lineage Assurance for Non-Human Identities in Critical Multi-Agent Systems. *arXiv:2509.18415*, 2025.
- [33] H. S. Nirban, B. Hinduja, B. Mangal, V. Chakka, Y. Sharma. A Zero-Trust Identity Framework for Autonomous AI Agents: Cryptographic Attestation and Policy-Driven Authorization. In *Advanced Information Networking and Applications (AINA), Lecture Notes on Data Engineering and Communications Technologies*, pp. 258-269, 2026. DOI 10.1007/978-3-032-23311-0_22.
- [34] E. Neelou et al. A2AS: Agentic AI Runtime Security and Self-Defense. *arXiv:2510.13825*, 2025.
- [35] A. Gupta. Verifiability-First Agents: Provable Observability and Lightweight Audit Agents. *arXiv:2512.17259*, 2025.
- [36] I. David, M. Guarnieri, A. Gervais. Alignment Contracts for Agentic Security Systems. *arXiv:2605.00081*, 2026.
- [37] F. B. Schneider. Enforceable Security Policies. *ACM Transactions on Information and System Security*, 3(1):30-50, 2000. DOI 10.1145/353323.353382.
- [38] Z. Ding et al. ACLE-MCP: Attested Capability Leases for Execution-Time Trust in Remote LLM Tool Use. *arXiv:2609.02690*, 2026.
- [39] W. Zhao et al. ClawGuard: A Runtime Security Framework for Tool-Augmented LLM Agents Against Indirect Prompt Injection. *arXiv:2604.11790*, 2026.
- [40] Y. Wang et al. AgentWatcher: A Rule-Based Prompt Injection Monitor. *arXiv:2604.01194*, 2026.
- [41] G. Coker, J. Guttman, P. Loscocco, A. Herzog, J. Millen, B. O' Hanlon, J. Ramsdell, A. Segall, J. Sheehy, B. Sniffen. Principles of Remote Attestation. *International Journal of Information Security*, 10(2):63-81, 2011. DOI 10.1007/s10207-011-0124-7.

- [42] D. Dolev, A. C. Yao. On the Security of Public Key Protocols. *IEEE Transactions on Information Theory*, 29(2):198-208, 1983. DOI 10.1109/TIT.1983.1056650.
- [43] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, B.-Y. Yang. High-Speed High-Security Signatures. *Journal of Cryptographic Engineering*, 2(2):77-89, 2012. DOI 10.1007/s13389-012-0027-1.
- [44] S. Josefsson, I. Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032, IETF, 2017. DOI 10.17487/RFC8032.
- [45] A. Rundgren, B. Jordan, S. Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, IETF, 2020. DOI 10.17487/RFC8785.
- [46] M. El-Hindi, T. Ziegler, M. Heinrich, A. Lutsch, Z. Zhao, C. Binnig. Benchmarking the Second Generation of Intel SGX Hardware. In *Proceedings of the 18th International Workshop on Data Management on New Hardware (DaMoN)*, pp. 1-8, 2022. DOI 10.1145/3533737.3535098.